E-COMMERCE PLATFORM WITH AUGMENTED REALITY

By

Taha Fayyaz

Supervised By

Dr. Akmal Saeed Khattak

Department of Computer Sciences

Quaid-i-Azam University, Islamabad

(2022-2026)

CERTIFICATE OF APPROVAL

Supervisor: _____

Dr. Akmal Saeed Khattak

Department of Computer Sciences

Quaid-i-Azam University Islamabad.

Chairperson: _____

Prof. Dr. Ayyaz Hussain

Department of Computer Sciences

Quaid-i-Azam University Islamabad.

Date: _____

ACKNOWLEDGEMENT

First and foremost, all praise is due to Almighty Allah, who blessed me with the strength, knowledge, and patience to carry out and complete this project successfully.

I would like to express my sincere gratitude to all my respected teachers Dr. Ayyaz Hussain, Dr. Rabeeh Ayaz Abbasi, Dr. Onaiza Maqbool, Dr. Mudassar Azam Sindhu, Dr. Khalid Saleem, Dr. Ghazanfar Farooq, Dr. Muhammad Naqi, Dr. Shuaib Karim, Dr. Umer Rasheed, Dr. Muazzam Khattak, Miss Ifrah Farrukh, and Miss Memoona Afsheen, for their consistent guidance and support throughout my academic journey. Their dedication to teaching and mentoring has played a vital role in developing my understanding and skills in computer science.

I am especially grateful to my supervisor, Dr. Akmal Saeed Khattak, whose continuous guidance, valuable suggestions, and constructive feedback have been central to the successful completion of this project.

I would also like to appreciate my friends Wajeeha Kamran, Tayyab Mehboob and Muhammad Hamad for their constant encouragement, moral support, and companionship during this journey.

Finally, my deepest thanks go to my family for their unwavering support, prayers, and patience throughout my years of study and during the completion of this project.

ABSTRACT

With rapid growth of online shopping, users increasingly struggle to visualize how products will look and fit in their real environment. Items such as rugs, furniture, home decorations and accessories are often purchased based on static images and dimensions, which can lead to confusion and product returns when the item that is delivered does not match the user expectations. Traditional E-commerce interfaces do not provide an immersive aware review of the product, which limits the customer's ability to make informed decisions.

This project proposes a mobile-based e-commerce platform enhanced with Augmented Reality (AR) to provide users with a more realistic and interactive shopping experience. This system allows the users to browse products, view detailed information and then virtually place 3D models of selected items into their surroundings using the camera of their smartphone. The application is developed using Flutter for cross-platform mobile development, which is integrated with a cloud backend for user authentication, product and order data management, and media storage. The functionality of AR is enabled through an AR framework that leverages device capabilities such as surface detection and real-time rendering of 3D assets.

By enabling users to preview the products in their own environment before purchasing, the system aims to increase user confidence, improve decision making and reduce the likelihood of returns. The platform serves as a smart, user-friendly tool that combines e-commerce and AR technologies to create a more engaging and trustworthy online shopping experience.

ACKNOWLEDGEMENT i
ABSTRACT ii
LIST OF FIGURES vii
LIST OF TABLES viii

Table of Contents

CHAPTER # 1 SOFTWARE PROJECT MANAGEMENT PLAN (SPMP).....

1.1 INTRODUCTION 2

1.1.1 Existing E-Commerce Systems With AR.....3

1.1.2 Existing E-Commerce Systems Without AR.....4

1.1.3 Project Overview.....5

1.1.4 Project Deliverables 5

1.2 PROJECT ORGANIZATION 5

1.2.1 Software Process Model.....5

1.2.2 Roles and Responsibilities 6

1.2.3 Tools and Techniques 7

1.3 PROJECT MANAGEMENT PLAN 7

1.3.1 Tasks.....7

1.3.2 Description.....8

1.3.3 Project Plan 9

1.3.4 Project Timeline 10

CHAPTER # 2 SOFTWARE REQUIREMENTS SPECIFICATIONS (SRS) .....

2.1 INTRODUCTION.....12

2.1.1 Problem Statement 12

2.1.2 Motivation 12

2.1.3 Objectives 12

2.1.4 Scope 13

2.1.5 Constraints 13

2.1.6 Assumptions 13

2.1.7 System Workflow Diagram.....14

2.2 OVERALL DESCRIPTION 15

2.2.1 Product Perspective.....15

2.2.2 User Characteristics 15

2.3 SPECIFIC REQUIREMENTS.....16

2.3.1 External Interface Requirements.....16

2.3.2 Functional Requirements 17

2.3.3 Non-Functional Requirements 18

2.4 SOFTWARE PRODUCT FEATURES 19
2.5 SOFTWARE SYSTEM ATTRIBUTES 19
2.5.1 Reliability 19
2.5.2 Availability 20
2.5.3 Security 20
2.5.4 Maintainability 20
2.5.5 Portability 20
2.5.6 Performance 20
2.6 DATABASE REQUIREMENTS 20
2.7 USE CASES 21
2.7.1 Use Cases List 21
2.7.2 Use Case Diagram 22
2.7.3 Use Cases Description 23
2.8 SYSTEM SEQUENCE DIAGRAMS 31
2.8.1 SSD-1: Create Customer Account 32
2.8.2 SSD-2: Explore Product Catalog 32
2.8.3 SSD-3: Review Product Details 33
2.8.4 SSD-4: Visualize Product in Augmented Reality 34
2.8.5 SSD-5: Maintain Shopping Cart 34
2.8.6 SSD-6: Checkout and Submit Order 35
2.8.7 SSD-7: Review Order History 36
2.8.8 SSD-8: Maintain Customer Profile 37
2.8.9 SSD-9: Maintain Product Catalog 37
2.8.10 SSD-10: Maintain Product Media Assets 38
2.8.11 SSD-11: Maintain Inventory/Stock 38
2.8.12 SSD-12: Process Orders 39
2.9 DOMAIN MODEL 39
CHAPTER # 3 SOFTWARE DESIGN DESCRIPTION (SDD) .....
3.1 INTRODUCTION 42
3.1.1 Design Overview 42
3.1.2 Requirements Traceability Matrix 42
3.2 SYSTEM ARCHITECTURAL DESIGN 43
3.2.1 Chosen System Architecture 43
3.2.2 Discussion of Alternative Designs 44
3.2.3 System Interface Description 45
3.3 DETAILED DESCRIPTION OF COMPONENTS 45
3.3.1 User Interface (Presentation Layer) 46

3.3.2 Controller and State Management Component 46
3.3.3 Authentication and Authorization Component 46
3.3.4 Catalog and Product Management Component 46
3.3.5 AR Preview Component 46
3.3.6 Cart Management Component 46
3.3.7 Checkout, Order Processing, and Payment Component 47
3.3.8 Data Access and Media Storage Component (Data Layer) 47
3.4 ARCHITECTURAL DIAGRAM 47
3.5 SEQUENCE DIAGRAMS 48
3.5.1 SD-1: Create Customer Account 48
3.5.2 SD-2: Explore Product Catalog 48
3.5.3 SD-3: Review Product Details 49
3.5.4 SD-4: Visualize Product in Augmented Reality 50
3.5.5 SD-5: Maintain Shopping Cart 50
3.5.6 SD-6: Checkout and Submit Order 51
3.5.7 SD-7: Review Order History 52
3.5.8 SD-8: Maintain Customer Profile 52
3.5.9 SD-9: Maintain Product Catalog 53
3.5.10 SD-10: Maintain Product Media Assets 53
3.5.11 SD-11: Maintain Inventory/Stock 54
3.5.12 SD-12: Process Orders 54
3.6 CLASS DIAGRAM 55
CHAPTER # 4 SOFTWARE TEST DOCUMENTATION (STD) .....
4.1 INTRODUCTION 58
4.1.1 System Overview 58
4.1.2 Test Approach 58
4.2 TEST PLAN 58
4.2.1 Features to be Tested 58
4.2.2 Features not to be Tested 59
4.2.3 Testing Tools and Environment 59
4.3 TEST CASES 59
4.3.1 TC-1: Registration Validation (Valid/Duplicate/Invalid) 59
4.3.2 TC-2: Catalog Load + Search/Filter Accuracy 60
4.3.3 TC-3: Product Detail Integrity (Price/Media/Stock/AR Flag) 60
4.3.4 TC-4: AR Preview Launch (Permission/Device/Model Cases) 61
4.3.5 TC-5: Cart Operations (Add/Update/Remove + Totals + Stock Limit) 62
4.3.6 TC-6: Checkout Order Creation + Stock Recheck + Inventory Decrement 63

4.3.7 TC-7: Order History Access Control + Listing 63
4.3.8 TC-8: Profile Update Validation + Persistence 64
4.3.9 TC-9: Admin Product CRUD + Field Validation 64
4.3.10 TC-10: Admin Media Upload + Product Linking 65
4.3.11 TC-11: Admin Stock Update Rules (Non-negative + Reflected) 66
4.3.12 TC-12: Admin Order Status Update + Customer Visibility 66
REFERENCES 68

List of Figures

Figure 1.1 : E-Commerce with Augmented Reality (AR) 2
Figure 1.2 : IKEA Place / IKEA App 3
Figure 1.3 : Amazon AR View 4
Figure 1.4 : Houzz App 4
Figure 1.5 : Iterative Incremental Software Process Model 5
Figure 1.6 : Project Management Plan 9
Figure 1.7 : Project Management Timeline 10
Figure 2.1 : System Workflow Diagram 14
Figure 2.2 : Use Case Diagram 22
Figure 2.3 : SSD-1 Create Customer Account 32
Figure 2.4 : SSD-2 Explore Product Catalog 33
Figure 2.5 : SSD-3 Review Product Details 33
Figure 2.6 : SSD-4 Visualize Product in AR 34
Figure 2.7 : SSD-5 Maintain Shopping Cart 35
Figure 2.8 : SSD-6 Checkout and Submit Order 36
Figure 2.9 : SSD-7 Review Order History 36
Figure 2.10 : SSD-8 Maintain Customer Profile 37
Figure 2.11 : SSD-9 Maintain Product Catalog 37
Figure 2.12 : SSD-10 Maintain Product Media Assets 38
Figure 2.13 : SSD-11 Maintain Inventory/Stock 39
Figure 2.14 : SSD-12 Process Orders 39
Figure 2.15 : Domain Model 40
Figure 3.1 : Three-Tier Layered Architecture Diagram 47
Figure 3.2 : SD-01 Create Customer Account 48
Figure 3.3 : SD-02 Explore Product Catalog 49
Figure 3.4 : SD-03 Review Product Details 49
Figure 3.5 : SD-04 Visualize Product in AR 50
Figure 3.6 : SD-05 Maintain Shopping Cart 51
Figure 3.7 : SD-06 Checkout and Submit Order 51
Figure 3.8 : SD-07 Review Order History 52
Figure 3.9 : SD-08 Maintain Customer Profile 52
Figure 3.10 : SD-09 Maintain Product Catalog 53
Figure 3.11 : SD-10 Maintain Product Media Assets 54
Figure 3.12 : SD-11 Maintain Inventory/Stock 54
Figure 3.13 : SD-11 Process Orders 55
Figure 3.14 : Class Diagram 56

List of Tables

Table 2.1 : Customer Use Cases 21
Table 2.2 : Admin Use Cases 21
Table 2.3 : UC-1 Create Customer Account 23
Table 2.4 : UC-2 Explore Product Catalog (Browse/Search/Filter) 24
Table 2.5 : UC-3 Review Product Details 25
Table 2.6 : UC-4 Visualize Product in Augmented Reality 26
Table 2.7 : UC-5 Maintain Shopping Cart 26
Table 2.8 : UC-6 Checkout and Submit Order 28
Table 2.9 : UC-7 Review Order History 28
Table 2.10 : UC-8 Maintain Customer Profile 29
Table 2.11 : UC-9 Maintain Product Catalog 29
Table 2.12 : UC-10 Maintain Product Media Assets 30
Table 2.13 : UC-11 Maintain Inventory/Stock 31
Table 2.14 : UC-12 Process Orders 31
Table 3.1 : Requirements Traceability Matrix 43
Table 4.1 : TC-01 Registration Validation (Valid/Duplicate/Invalid) 60
Table 4.2 : TC-02 Catalog Load + Search/Filter Accuracy 60
Table 4.3 : TC-03 Product Detail Integrity (Price/Media/Stock/AR Flag) 61
Table 4.4 : TC-04 AR Preview Launch (Permission/Device/Model Cases) 62
Table 4.5 : TC-05 Cart Operations 62
Table 4.6 : TC-06 Checkout Order Creation 63
Table 4.7 : TC-07 Order History Access Control + Listing 64
Table 4.8 : TC-08 Profile Update Validation + Persistence 64
Table 4.9 : TC-09 Admin Product CRUD + Field Validation 65
Table 4.10 : TC-10 Admin Media Upload + Product Linking 65
Table 4.11 : TC-11 Admin Stock Update Rules 66
Table 4.12 : TC-12 Admin Order Status Update 67

CHAPTER # 1

SOFTWARE PROJECT MANAGEMENT PLAN (SPMP)

1.1 INTRODUCTION

E-commerce has become a common way for people to buy products, especially for categories such as clothes, furniture, rugs, home decorations and accessories. Online platforms allow the users to compare prices, browse catalogues and place orders without visiting a physical store. However, customers usually see only static images, written descriptions and a few user reviews. For the products that are heavily on size, color and placement in a room, it is still difficult to judge how an item will look and fit in the user's own environment, which can lead to uncertainty and product returns.

Figure 1.1 provides a visual representation of how AR technology improves the online shopping experience by allowing users to visualize the products within their own environment before purchase, reducing uncertainty and likelihood of returns.

Figure 1.1 : E-Commerce with Augmented Reality (AR)

Augmented Reality (AR) helps reduce this gap between the online catalogue and the real world. AR systems overlay digital content, such as 3D objects or labels on top of live camera view of a phone or tablet, allowing the users to see virtual items inside their real surroundings. This technology is already used in social media filters, navigation, games and education to make information more interactive and context aware.

When AR is combined with e-commerce, customers now can visualize products at real scale in their own rooms before buying. For example, a user can place a virtual sofa on the living room floor, move it around and see how it looks with existing furniture and wall colors. Industry reports indicate that such AR experiences can increase user engagement, support more confident purchase decisions and help reduce the likelihood of returns.

1.1.1 Existing E-Commerce Systems

1.1.1 Existing E-Commerce Systems With AR

Several well-known e-commerce applications already include AR features for furniture and home décor:

- **IKEA PLACE / IKEA APP**: IKEA's AR solution allows users to place true to scale 3D models of IKEA furniture in their rooms using a smartphone camera. It is limited to IKEA products and markets where IKEA operates.

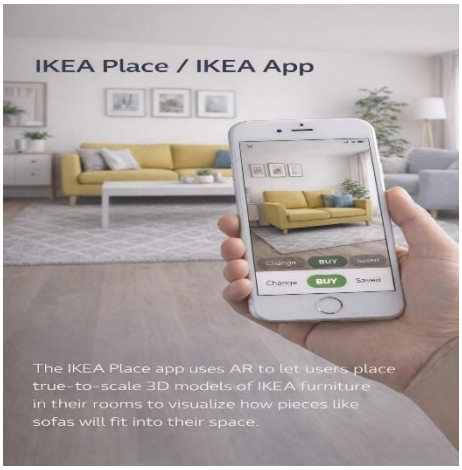

Figure 1.2 : IKEA Place / IKEA App

Figure 1.2 shows The IKEA Place app uses AR to let users place true-to-scale 3D models of IKEA furniture in their rooms to visualize how pieces like sofas will fit into their space.

- **AMAZON AR VIEW**: The Amazon app offers an AR mode for selected items, enabling the users to see certain items inside their own environment. It is only available for a subset of products.

Figure 1.3 : Amazon AR View

Figure 1.3 shows that it allows users to visualize selected products in their own space at true scale using AR. In this figure, a person is holding a smartphone to view a brown armchair placed within a real living room setting.

- **HOUZZ**: This app includes AR tools for visualizing tiles, décor and other items on floors and walls under real lighting conditions. It supports only selected product types sold through Houzz.

Figure 1.4 : Houzz App

Figure 1.4 shows this app uses AR to let users visualize furniture and décor in their own space. In this figure, a person is holding a smartphone displaying a 3D AR model of a modern living room setup, including a navy sectional sofa and a round wooden coffee table.

1.1.2 Existing E-Commerce Systems Without AR

Traditional e-commerce platforms such as Amazon, Daraz, AliExpress and many local online stores mainly reply on 2D images, text descriptions, customer reviews and specifications. They are effective for listing large catalogues and managing orders, but they do not allow users to visualize products inside their own rooms. Because of this customer may still struggle to understand the true scale and appearance of items which

can result in incorrect size expectations, difficulty in matching colors and a string dependence on guesswork and reviews instead of visual confirmation.

1.1.3 Project Overview

The main goal of this project is to develop a mobile e-commerce application with built-in AR visualization features. The app will allow the users to browse different product categories (such as clothes, furniture, carpets, home decorations and accessories), view detailed product information and then use AR to virtually place selected items in their real environment using the phone's camera.

The system will be implemented using Flutter for cross-platform mobile development. The backend will use Firebase services, including Firebase Authentication for user accounts, Cloud Firestore for storing products, user and order data and Firebase Storage for product images and 3D AR models.

The key features of the project include:

- User registration and login
- Browse the product and search it by category
- Product details with images, description, price and AR preview option
- Shopping cart and order placement
- AR based visualization of selected products in the user's environment
- Order history and profile management

1.1.4 Project Deliverables

The deliverables of my project are as follows:

- Software Project Management Plan
- Software Requirements Specification
- Software Design Description
- Software Text Documentation
- Mobile Application Interfaces
- Augmented Reality Visualization Module
- Backend Configuration
- Source Code
- Final Working Software (demonstrating AR-enabled e-commerce flow)

1.2 PROJECT ORGANIZATION

1.2.1 Software Process Model

I chose the Iterative Incremental Model for development of the E-Commerce Platform with Augmented Reality due to the following reasons

- The system consists of multiple modules (product catalog, authentication, AR viewer) that can be developed and improved in small increments.
- AR features often require experimentation and tuning, which fits well with an iterative approach where feedback is used to refine each increment.
- The Iterative Incremental Model allows delivering a basic working e-commerce app in early iterations (without AR) and then gradually integrating and improving the AR module in later iterations.

As shown in *Figure 1.5,* the Iterative Incremental Software Process Model organizes the project into repeated cycles. Each iteration delivers a working subset of the system and later iterations add more functionality and refinement.

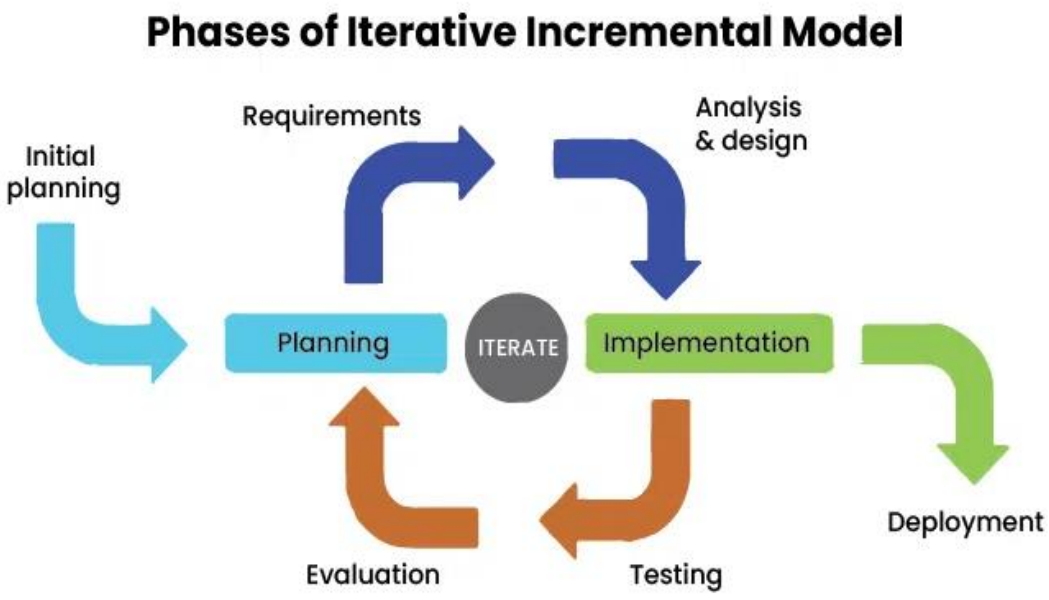

Figure 1.5 : Iterative Incremental Software Process Model

1.2.2 Roles and Responsibilities

Developer (Student):

- Select and study suitable AR tools and Flutter plugins for integrating ARCore with the mobile app.
- Design and implement the Flutter UI for product browsing, AR preview, cart and orders.
- Configure and integrate Firebase services (Authentication, Firestore, Storage) with the mobile app.

- Implement core e-commerce logic, including product management, cart operations and order handling.
- Implement the AR visualization module, ensuring that the models can be placed, moved and viewed realistically in the user's environment.
- Perform unit and integration testing of the critical features and fix bugs during development.

Supervisor:

- Provide guidance on the overall system design, especially the use of AR for e-commerce and the choice of technologies.
- Review the project documents and provide feedback timely.
- Help resolve conceptual or technical issues when the student faces major roadblocks.

1.2.3 Tools and Techniques

Tools Used:

- Flutter and Dart for developing mobile application
- Android Studio as an IDE
- Firebase Authentication, Cloud Firestore, Firebase Storage for backend services
- AR Flutter Plugin to enable augmented reality
- ProjectLibre for project scheduling, task planning and Gantt Chart generation
- Visual Paradigm / draw.io for drawing UML diagrams
- Git and GitHub for version control and backups
- MS Word with IEEE-style format for preparing project documentation

Techniques Used:

- Iterative and Incremental Development
- Modular Design
- User-Centered UI Design
- AR Plane Detection and Object Placement
- State Management in Flutter
- Basic Security Practices

1.3 PROJECT MANAGEMENT PLAN

1.3.1 Tasks

The project consists of the following main phases:

- Requirements Analysis Phase
- System Design Phase
- Prototype and UI Design Phase
- Development Phase
- Testing & Validation Phase

1.3.2 Description

The first step is the Requirements Analysis which helps identify key functional and non-functional requirements of the AR enabled e-commerce mobile application, such as product browsing, AR visualization, cart management and performance constraints on mobile devices.

The next phase is System Design Phase where the overall architecture is defined, the interaction between the mobile app and backend is planned, data models are designed and the AR module's integration points are specified.

The Prototype and UI Design then focus on creating and refining the main app screens in Flutter, designing user flows for browsing products, viewing details, launching AR view and placing orders so that the interface is initiative and user-friendly.

In the Development Phase, the planned features are implemented in iterations, including user authentication, product catalog, AR product placement using device camera, cart operations and order handling.

Testing and Validation phase is concerned with checking the correctness and reliability of all the modules through functional testing, AR behavior testing on real devices, usability feedback from users and fixing issues to ensure that the system meets the defined requirements before the deployment.

1.3.3 Project Plan

Figure 1.6 presents the project plan in the form of Gantt chart. The chart groups tasks into requirement analysis, system design, prototype and UI design, development, testing and validation activities and shows how these tasks are scheduled over the documentation phase of the project.

		Name	Duration	Start	Finish
1		**Planning**	**10 days**	**10/1/25 8:00 AM**	**10/14/25 5:00 PM**
2		Define Project Deliverables for AR E-Commerce App	2 days	10/1/25 8:00 AM	10/2/25 5:00 PM
3		Choose & Define Software Process Model	2 days	10/3/25 8:00 AM	10/6/25 5:00 PM
4		Define Roles & Responsibilities	2 days	10/7/25 8:00 AM	10/8/25 5:00 PM
5		Define Tools & Techniques (Flutter, Firebase, AR plugin, etc.)	2 days	10/9/25 8:00 AM	10/10/25 5:00 PM
6		Define Project Timeline (docs + implementation)	1 day	10/13/25 8:00 AM	10/13/25 5:00 PM
7		Verify SPMP	1 day	10/14/25 8:00 AM	10/14/25 5:00 PM
8		**Analysis**	**15 days**	**11/3/25 8:00 AM**	**11/21/25 5:00 PM**
9		Describe Scope & Objectives of AR E-Commerce System	2 days	11/3/25 8:00 AM	11/4/25 5:00 PM
10		Define Functional & Non-Functional Requirements	4 days	11/5/25 8:00 AM	11/10/25 5:00 PM
11		Define Use Cases (customer, admin, AR preview, orders, etc.)	3 days	11/11/25 8:00 AM	11/13/25 5:00 PM
12		Draw Use Case Diagram	2 days	11/14/25 8:00 AM	11/17/25 5:00 PM
13		Draw System Sequence Diagrams	2 days	11/18/25 8:00 AM	11/19/25 5:00 PM
14		Draw Domain Model	1 day	11/20/25 8:00 AM	11/20/25 5:00 PM
15		Verify SRS	1 day	11/21/25 8:00 AM	11/21/25 5:00 PM
16		**Software Design**	**20 days**	**12/1/25 8:00 AM**	**12/26/25 5:00 PM**
17		Create Requirements Traceability Matrix	2 days	12/1/25 8:00 AM	12/2/25 5:00 PM
18		Design System Architecture (Flutter app + Firebase + AR module)	3 days	12/3/25 8:00 AM	12/5/25 5:00 PM
19		Describe System Interfaces (screens, API, Firebase collections, AR)	3 days	12/8/25 8:00 AM	12/10/25 5:00 PM
20		Describe System Components (UI, business logic, AR layer, backend)	3 days	12/11/25 8:00 AM	12/15/25 5:00 PM
21		Draw Sequence Diagrams (key flows: browse AR cart order)	3 days	12/16/25 8:00 AM	12/18/25 5:00 PM
22		Draw Class Diagram	2 days	12/19/25 8:00 AM	12/22/25 5:00 PM
23		Design User Interfaces (Flutter screen layouts for AR e-commerce)	3 days	12/23/25 8:00 AM	12/25/25 5:00 PM
24		Verify SDD	1 day	12/26/25 8:00 AM	12/26/25 5:00 PM
25		**Testing and Validation**	**15 days**	**1/15/26 8:00 AM**	**2/4/26 5:00 PM**
26		Define Test Approach Used (black-box, device testing, AR checks)	2 days	1/15/26 8:00 AM	1/16/26 5:00 PM
27		Make Test Plan (features to test, environment, devices, schedule)	3 days	1/19/26 8:00 AM	1/21/26 5:00 PM
28		Write Test Cases (login, catalog, AR placement, cart, orders, etc.)	7 days	1/22/26 8:00 AM	1/30/26 5:00 PM
29		Verify STD	3 days	2/2/26 8:00 AM	2/4/26 5:00 PM

Figure 1.6 : Project Management Plan

1.3.4 Project Timeline

Figure 1.7 summarizes the overall timeline of the project from the start of documentation work in October 2025 to the completion of implementation and testing in June 2026. It highlights the separation between documentation-focused period and later implementation-focused period.

Predecessors	Resource Names
	MS Word;Taha Fayyaz
2	MS Word;Taha Fayyaz
3	MS Word;Taha Fayyaz
4	MS Word;Taha Fayyaz
5	ProjectLibre;Taha Fayyaz
6	Dr. Akmal Saeed Khattak
1	
7	MS Word;Taha Fayyaz
9	MS Word;Taha Fayyaz
10	MS Word;Taha Fayyaz
11	Taha Fayyaz;Visual Paradigm
12	Taha Fayyaz;Visual Paradigm
13	Taha Fayyaz;Visual Paradigm
14	Dr. Akmal Saeed Khattak
8	
15	MS Word;Taha Fayyaz
17	Taha Fayyaz;Visual Paradigm
18	MS Word;Taha Fayyaz
19	MS Word;Taha Fayyaz
20	Taha Fayyaz;Visual Paradigm
21	Taha Fayyaz;Visual Paradigm
22	Figma;Flutter UI;Taha Fayyaz
23	Dr. Akmal Saeed Khattak
16	
24	MS Word;Taha Fayyaz
26	MS Word;Taha Fayyaz
27	MS Word;Taha Fayyaz
28	Dr. Akmal Saeed Khattak

Figure 1.7 : Project Management Timeline

CHAPTER # 2

SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

2.1 INTRODUCTION

This chapter defines the software requirements for E-Commerce Platform with Augmented Reality. It clarifies the problem being solved, why the system is needed, what the project intends to deliver, and the boundaries under which the system will operate. The chapter also states key constraints and assumptions and presents the overall workflow of the proposed system. This chapter also presents the system use cases and their descriptions for clarity.

2.1.1 Problem Statement

Conventional E-commerce applications rely on static product images, basic dimensions, and text descriptions. For items such as clothes, furniture, rugs and home decorations, this makes it difficult for users to estimate true scale, placement and visual fit in their own environment. As a result, customers make uncertain decisions, leading to dissatisfaction and higher return rates.

The problem is to develop an e-commerce system that allows users to preview products in their real surroundings before purchase using Augmented Reality, while still supporting standard shopping features (browse cart, checkout and order history).

2.1.2 Motivation

The motivation for this project is to reduce the guesswork in online shopping for environment-dependent products. AR based visualization can improve user experience and confidence by enabling a realistic preview at near-actual scale and placement. A more informed decision process is expected to:

- Increase user trust in product selection.
- Improve shopping engagement.
- Reduce avoidable returns caused by size/fit mismatch.
- Deliver a modern and competitive shopping experience on mobile devices.

2.1.3 Objectives

The main objectives of the project are:

- Develop a mobile-based e-commerce application for product browsing and purchasing.
- Provide an AR preview feature to place products 3D model into user's real environment.
- Support core shopping flows: authentication, product browsing/search, product details, cart, checkout and order placement.
- Store and manage product, user and order data using cloud backend (Firebase services).
- Maintain a user-friendly interface with stable performance on supported mobile devices.

2.1.4 Scope

The scope of this project includes the development of a cross-platform mobile application with the following coverage:

- User registration, login and profile management.
- Product catalog browsing and category-based navigation.
- Product detail view (images, description, price and AR preview option).
- AR-based visualization for products.
- Cart management and checkout flow.
- Order placement and order history view.
- Cloud data storage for product images, product metadata and 3D assets.

2.1.5 Constraints

The system is subject to the following constraints:

- **Device Compatibility:** AR preview requires AR-supported devices (AR Core/ARKit capable). Devices without AR support may use the app but cannot use AR preview.
- **Performance:** Real-time AR rendering depends on device GPU; low-end devices may show reduced performance.
- **Network Dependence:** Internet connectivity is required to fetch product catalogs, images and 3D models from cloud storage.
- **3D Asset Availability:** AR preview is only possible for products that have valid, optimized 3D models uploaded to storage.
- **Lighting/Surface Limitations:** AR plane detection depends on environment lighting and visible surfaces; inaccurate detection may affect preview quality.
- **Project Timeline:** Development and documentation must follow the semester/FYP schedule, limiting scope expansion.

2.1.6 Assumptions

The project is designed under these assumptions:

- Users have access to a smartphone with a functional camera and basic mobile usage skills.
- Users have stable internet for typical browsing and checkout flows.
- Product catalog data (price, description, images, and stock availability) is maintained correctly in the backend.
- 3D models are provided in supported formats and are optimized for mobile rendering.
- Users will use AR preview in suitable conditions (adequate lighting and visible floor/table surface).
- The system targets integration with a third-party payment gateway for online payments. If gateway integration is not completed in the prototype, the system will support Cash-on-Delivery (COD) or simulated payment confirmation.

2.1.7 System Workflow Diagram

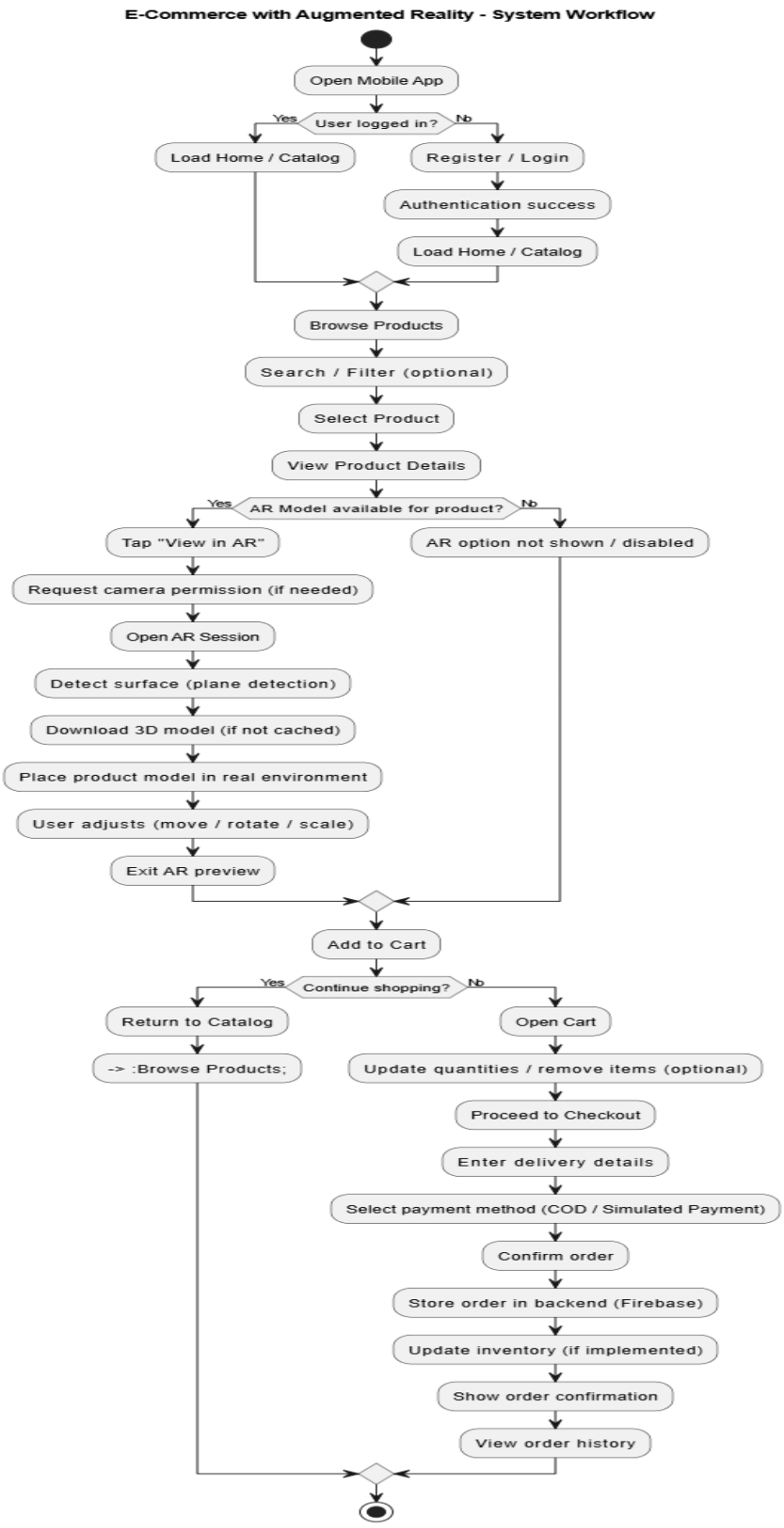

Figure 2.1 : System Workflow Diagram

Figure 2.1 shows how the system operates from start to order completion. The user opens the app, logs in (if required) and browses the product catalog using search/filter options. After viewing product details, the user can preview the product in AR when a 3D model is available; otherwise, the AR option remains disabled. The user then adds the product to the cart, proceeds to checkout, enters delivery details, selects a payment method and confirms the order. Finally, the order is stored in the backend (Firebase) and the user can view confirmation and order history.

2.2 OVERALL DESCRIPTION

2.2.1 Product Perspective

The E-Commerce Platform with Augmented Reality is a mobile-based standalone system that integrates standard e-commerce operations with an AR visualization module. The system allows customers to browse products, view detailed information, and preview products in their real environment using AR, and place orders through a checkout process. A dedicated Admin Interface is also included to manage product data, AR assets, and order records. The solution relies on cloud services for authentication, data storage and media hosting.

Relationship with external components/systems:

- **Firebase Authentication (External Service):** Used for user registration, login and session management.
- **Cloud Firestore (External Database Service):** Stores structured data including user profiles, product catalog details, cart/order records and order status.
- **Firebase Storage (External Storage Service):** Stores product images and 3D models required for AR-enabled products.
- **AR Framework (ARCore/ARKit via AR plugin):** Handles AR session creation, plane detection, tracking and rendering of 3D product models in the real environment.
- **Device Hardware (Camera and Sensors):** The mobile camera and sensors are required to support AR preview and environment tracking.
- **Payment Gateway (External Service):** Used to authorize/capture online payments. COD/simulated confirmation may be used as a prototype fallback.

Internal module included in the product:

- **Admin Interface Module:** A dedicated interface for the administrator to manage products, upload images and 3D models and monitor/update orders. It connects to the same cloud backend (Firestore/Storage) used by the customer application.

2.2.2 User Characteristics

1. **Customer / Buyer (Primary User):**
   - Can register/login, browse products, search/filter, view product details, and place orders.

- Uses AR preview for products to visualize product placement and approximate size before purchase.
- Requires a simple and guided checkout flow with minimal steps.

2. **Administrator (Admin Interface User):**
   - Use the admin interface to manage system data and operations.
   - Can add/update/delete products and upload/associates 3D models with products for AR preview.
   - Uploads product images and upload/associates 3D models with products for AR preview.
   - Can view orders and update status (pending/confirmed/shipped/delivered) based on the implemented workflow.

2.3 SPECIFIC REQUIREMENTS

2.3.1 External Interface Requirements

2.3.1.1 User Interfaces

- **Customer Mobile Application:**
  - The system shall provide Login/Signup interfaces with validation for required fields.
  - The system shall provide Catalog interface that lists products with at minimum: *name, price, thumbnail.*
  - The system shall provide Search by product name and Filter by category.
  - The system shall provide Product Detail interface showing: *images, name, price, description* and an AR Preview only when a 3D model is linked.
  - The system shall provide Cart interface showing items, quantities, subtotal and total.
  - The system shall provide Checkout interface to capture: *recipient name, phone, delivery address and payment option.*
  - The system shall provide Order Confirmation and Order History interfaces displaying Order ID, summary and status.

- **Admin Interface:**
  - The system shall provide Admin Login.
  - The system shall provide Product Management (add/update/delete).
  - The system shall provide Media Management to upload product images and upload/associate 3D models.
  - The system shall provide Order Management to view orders and update order status.

2.3.1.2 Hardware Interfaces

- **Customer side:** Android/IOS smartphone with camera.
- **AR Preview:** Device shall be ARCore/ARKit supported.
- **Admin Side:** Laptop/Desktop (or Mobile) capable of running the admin interface.

2.3.1.3 Software Interfaces

- **Mobile App:** Flutter (Dart).
- **Backend:** Firebase Authentication, Cloud Firestore, Firebase Storage.
- **AR:** ARCore/ARKit via Flutter AR plugin.

2.3.1.4 Communication Protocols

- The system shall use HTTPS for all client-backend communication.
- The system shall access images/3D models via Firebase Storage using security rules and authenticated access.

2.3.2 Functional Requirements

FR-01 User Authentication

- The system shall allow users to register and login using Authentication service.
- The system shall prevent access to cart/checkout/order history when the user is not authenticated.

FR-02 Product Catalog

- The system shall retrieve products from the product catalog and display them in the catalog.
- Each product shall include at minimum: *name, price, category, description,* stockQuantitiy*, images*.

FR-03 Search and Filter

- The system shall support keyword search on product name.
- The system shall support filtering by category.

FR-04 Product Details

- The system shall display full product details including images, price and description.
- The system shall display the AR Preview option only if the product has valid 3D model reference.
- The system shall display product availability as In Stock / Out of Stock based on stockQuantity.

FR-05 AR Session Start

- When AR Preview is selected, the system shall request camera permission if not granted.
- If permission is denied or device is unsupported, the system shall disable AR preview and display a reason.

FR-06 Cart Management

- The system shall allow users to add items to cart, update quantities, and remove items.
- The system shall compute subtotal and total based on cart contents.

- The system shall prevent users from adding quantity greater than the available stockQuantity.

FR-07 Checkout and Order Placement

- The system shall validate delivery fields (name, phone, address) before confirming an order.
- The system shall create and store an order record upon confirmation.
- The system shall verify stock availability before confirming an order.
- On successful order placement, the system shall decrease stockQuantity for each ordered product.

FR-08 Order History

- The system shall allow users to view only their own orders and current order status.

FR-09 Admin Product Management

- The system shall allow an administrator to add/update/delete products.
- The system shall allow admin to update stockQuantity for products.

FR-10 Admin Order Management

- The system shall allow admin to view all orders and update order status.

2.3.3 Non-Functional Requirements

Usability:

- The system shall allow a user to reach Product Details from Catalog within ≤ 3 taps.
- The system shall allow a user to complete (Add to Cart to Checkout to Confirm Order) within ≤ 6 taps (excluding typing).

Performance:

- The system shall load the initial Catalog screen within ≤ 5 seconds on a standard 4G/Wi-Fi connection.
- The system shall open Product Details within ≤ 4 seconds when data is cached, and ≤ 6 seconds when not cached.

Reliability:

- If a network request fails, the system shall display an error message within ≤ 10 seconds and provide a retry option.

Security:

- The system shall enforce HTTPS for all communication.
- Firestore rules shall ensure: users can read/write only their own profile/orders.
- Admin-only operations (product/order updates) shall be restricted to authorized admin accounts.

Compatibility:

- The system shall run on Android and IOS devices supported by the selected Flutter SDK.

- The system shall allow non-AR devices to use all e-commerce features except AR preview.

Maintainability:

- The system shall separate modules (auth, catalog, AR, cart, orders, admin) such that changes in one module do not require redesign of unrelated modules.

2.4 SOFTWARE PRODUCT FEATURES

The main software product features of this system include:

- **User Account Management**
  Registration, login/logout, and basic profile handling through cloud-based authentication.
- **Product Catalog Browsing**
  Product listing with category-wise navigation.
- **Search and Filtering**
  Keyword search (by product name) and category-based filtering to quickly locate items.
- **Product Detail View**
  Detailed product interface showing images, description, price, availability/stock status and AR availability indicator.
- **Augmented Reality (AR) Product Preview**
  AR visualization for products, allowing users to place a 3D model in the real environment.
- **Shopping Cart Management**
  Add-to-cart, quantity updates, item removal, and automatic subtotal/total calculation.
- **Checkout and Order Placement**
  Delivery details entry, payment option selection, order confirmation, and order record creation in the backend.
- **Order History and Tracking**
  View past orders and their current status.
- **Admin Interface for Product and Order Management**
  Admin can add/update/delete products, upload product images, upload/associate 3D models for AR, and view/update order statuses.

2.5 SOFTWARE SYSTEM ATTRIBUTES

2.5.1 Reliability

- The system shall preserve product, cart, and order data integrity during normal operation.
- The system shall ensure that confirmed orders are not lost once stored in Firestore.
- The system shall handle AR initialization or tracking failure gracefully by returning the user to the product interface with an explanatory message.

2.5.2 Availability

- The system shall be available 24/7 for customers, subject to Firebase service availability and internet connectivity.
- The admin interface shall be available during normal operational hours; however, it may also be hosted for 24/7 access if required by deployment.

2.5.3 Security

- The system shall use HTTPS for all client–backend communication.
- The system shall enforce Firebase security rules such that users can access only their own profile and orders, while admin privileges are required for product and order management.

2.5.4 Maintainability

- The system shall follow a modular structure (Authentication, Catalog, AR Preview, Cart, Orders, Admin).
- The system shall allow replacement or upgrade of AR plugins and Firebase configurations without requiring a full redesign of the application.

2.5.5 Portability

- The customer application shall be portable across Android and IOS platforms supported by the Flutter SDK.

2.5.6 Performance

- The system shall load the initial product catalog within ≤ 5 seconds on a standard 4G/Wi-Fi connection.

2.6 DATABASE REQUIREMENTS

The system uses Firebase Cloud Firestore as the primary database for structured data and Firebase Storage for media assets (product images and AR 3D models). The database is required to store application data for customers and administrators in a consistent and secure manner.

Firestore will store the following:

- **User Records**
  User ID, name, email, phone (optional), and basic profile fields.
- **Product Catalog**
  Product ID, title/name, category, price, description, image URLs, stockQuantity/ availability status and AR model reference (only for AR-supported products).
- **Cart Data**

Cart data shall be stored per user in Firestore to support session continuity.

- **Order Records**
  Order ID, user ID, ordered items, quantity, price totals, delivery details, timestamp, and order status (e.g., pending/confirmed/shipped/delivered).

Firebase Storage will store:

- **Product Images**
  (JPEG/PNG) linked through URLs stored in Firestore.
- **3D AR Models**
  linked through model references stored in Firestore.

Data retention and consistency:

- Orders are stored permanently for record and tracking purposes.
- Storage access shall be controlled through Firebase security rules so that customers cannot modify product catalog data, while admin can manage products and assets.

2.7 USE CASES

2.7.1 Use Cases List

Customer

Use Case ID	Use Case Name
UC-01	Create Customer Account
UC-02	Explore Product Catalog (Browse/Search/Filter)
UC-03	Review Product Details
UC-04	Visualize Product in Augmented Reality
UC-05	Maintain Shopping Cart
UC-06	Checkout and Submit Order
UC-07	Review Order History
UC-08	Maintain Customer Profile

Table 2.1: Customer Use Cases

Admin

Use Case ID	Use Case Name
UC-09	Maintain Product Catalog
UC-10	Maintain Product Media Assets (Images & 3D Models)
UC-11	Maintain Inventory/Stock
UC-12	Process Orders (View/Update Status)

Table 2.2: Admin Use Cases

2.7.2 Use Case Diagram

Figure 2.2 illustrates the major interactions between the system and its two primary actors: Customer and Administrator. The customer performs core shopping tasks including account creation, exploring the product catalog, reviewing product details, previewing items in augmented reality, managing the shopping cart, placing orders through checkout, viewing order history, and maintaining profile information. The administrator manages backend operations through product catalog maintenance, uploading product images and 3D models, updating inventory/stock, and processing customer orders by updating order status.

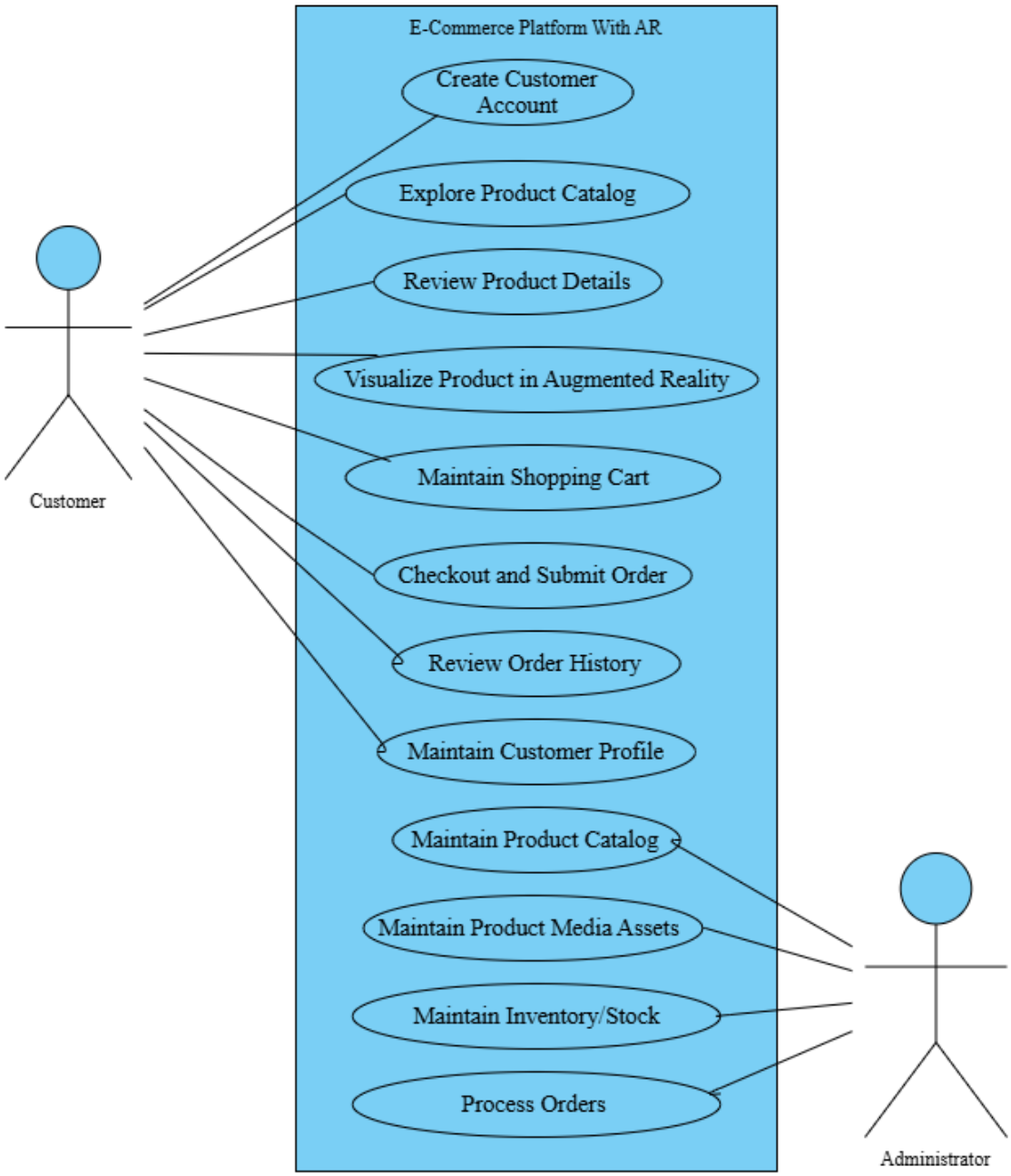

Figure 2.2: Use Case Diagram

2.7.3 Use Cases Description

UC-01 Create Customer Account

Table 2.3 describes how a customer creates an account by submitting registration details. The system validates the input, creates the account, initializes the customer profile, and grants access to the customer module.

Use Case ID	**UC-01**
Use Case Name	Create Customer Account
Primary Actor	Customer
Stakeholders & Interests	**Customer:** create account quickly and securely. **System Owner:** maintain valid user identities for ordering and tracking.
Preconditions	1. App is accessible. 2. Internet connectivity is available.
Postconditions	1. User account created. 2. Customer profile is initialized and linked to the account. 3. Customer session becomes active.
Main Success Scenario	1. Customer selects Sign Up. 2. System displays registration form. 3. Customer enters required details (name, email, password). 4. Customer submits the form. 5. System validates data and creates account. 6. System initializes the customer profile. 7. System redirects customer to Catalog/Home.
Extensions/Alternative Flows	3A Invalid data: System shows validation message; user retries. 5A Email already exists: System prompts login/reset. 5B Network/auth failure: System shows error and allows retry.
Frequency of Occurrence	Low (new customers).
Special Requirements	Password must be masked; error messages must be clear.
Technology	Flutter UI; Firebase Authentication; Firestore (Users).
Inputs	Name, email, password; submit action.
Outputs	1. Account created. 2. profile saved. 3. access to customer module.

Table 2.3: UC-1 Create Customer Account

UC-02 Explore Product Catalog

Table 2.4 describes how a customer explores products by browsing the catalog and applying search and category filters. The system retrieves matching products from the product catalog and displays them in the catalog view.

Use Case ID	**UC-02**
Use Case Name	Explore Product Catalog
Primary Actor	Customer
Stakeholders & Interests	**Customer:** find products quickly. **System Owner:** products must appear with correct price and stock status.
Preconditions	1. App is open. 2. Internet connectivity is available.
Postconditions	Matching product list is displayed based on browse/search/filter selection.
Main Success Scenario	1. Customer opens Catalog screen. 2. System fetches products. 3. Customer browses categories. 4. Customer enters a keyword and/or selects category filter. 5. System updates list based on criteria. 6. Customer selects a product.
Extensions/Alternative Flows	2A Load failure: System shows error and retry. 5A No matches: System shows (No products found).
Frequency of Occurrence	High.
Special Requirements	Catalog should load within defined performance limits.
Technology	Firestore queries; cached image URLs (Storage).
Inputs	Search keyword; selected category; browse actions.
Outputs	Updated catalog list.

Table 2.4: UC-2 Explore Product Catalog (Browse/Search/Filter)

UC-03 Review Product Details

Table 2.5 describes how a customer opens a product to view its details, including price, images, description, stock status, and AR availability.

Use Case ID	**UC-03**
Use Case Name	Review Product Details
Primary Actor	Customer
Stakeholders & Interests	**Customer:** understand product before purchase. **System Owner:** show correct stock and AR availability.
Preconditions	1. Product exists in catalog. 2. Customer selects a product.
Postconditions	1. Product details are displayed. 2. Stock and AR indicators are shown.
Main Success Scenario	1. Customer selects a product from catalog. 2. System retrieves product details. 3. System displays images, description, price. 4. System displays stock status (In Stock/Out of Stock). 5. System enables AR option only if model reference exists and device supports AR.

Extensions/Alternative Flows	2A Product removed/not found: System shows message and returns to catalog. 3A Media unavailable: System shows placeholder and continues.
Frequency of Occurrence	High.
Special Requirements	AR button must be disabled/hidden when unavailable or unsupported.
Technology	Firestore (Product); Storage URLs (images/model ref).
Inputs	Product selection.
Outputs	Product detail interface displayed.

Table 2.5: UC-3 Review Product Details

UC-04 Visualize Product in Augmented Reality

Table 2.6 describes how a customer previews a product using AR. The system starts an AR session, loads the 3D model, and renders it on a detected surface if the device supports AR and permissions are granted.

Use Case ID	**UC-04**
Use Case Name	Visualize Product in Augmented Reality
Primary Actor	Customer
Stakeholders & Interests	**Customer:** preview product at approximate scale/placement. **System Owner:** stable AR experience with clear failure handling.
Preconditions	1. Product has valid 3D model reference. 2. Device supports ARCore/ARKit. 3. Camera permission granted (or requestable).
Postconditions	AR preview shown OR user receives clear failure message and returns to product details.
Main Success Scenario	1. Customer taps View in AR. 2. System requests camera permission (if needed). 3. System initializes AR session. 4. System downloads/loads 3D model (cache if available). 5. System detects surface and displays placement guide. 6. Customer places and views model in environment. 7. Customer exits AR and returns to product details.
Extensions/Alternative Flows	2A Permission denied: System shows message; AR not started. 3A Unsupported device: System shows (AR not supported). 4A Model load fails: System shows error and returns to details. 5A Surface not detected: System shows guidance and allows retry/exit.
Frequency of Occurrence	Medium.

Special Requirements	AR failure must not crash; guidance must be visible if tracking is weak.
Technology	ARCore/ARKit via Flutter AR plugin; Firebase Storage model download.
Inputs	(View in AR) action; permission response.
Outputs	AR preview session OR explanatory error message.

Table 2.6: UC-4 Visualize Product in Augmented Reality

UC-05 Maintain Shopping Cart

Table 2.7 describes how a customer adds products to the cart, updates quantities, or removes items. The system recalculates totals and enforces stockQuantity limits.

Use Case ID	**UC-05**
Use Case Name	Maintain Shopping Cart
Primary Actor	Customer
Stakeholders & Interests	**Customer:** manage intended purchases. **System Owner:** enforce stock limits and maintain correct totals.
Preconditions	1. Customer is logged in. 2. Product exists in catalog.
Postconditions	1. Cart is updated and saved for the customer. 2. Subtotal/total updated.
Main Success Scenario	1. Customer taps Add to Cart on a product. 2. System checks stockQuantity. 3. System adds item (or increments quantity). 4. Customer opens Cart screen. 5. Customer updates quantity or removes item. 6. System recalculates totals and saves updated cart.
Extensions/Alternative Flows	2A Out of stock: System blocks add and shows message. 5A Quantity exceeds stock: System blocks update and shows limit. 6A Network failure: System shows error and retry; cart update not confirmed.
Frequency of Occurrence	High.
Special Requirements	Quantity updates must respect stockQuantity and never exceed it.
Technology	Firestore (Cart per user); Firestore (Product stockQuantity).
Inputs	Add/remove actions; quantity changes.
Outputs	Updated cart items and totals.

Table 2.7: UC-5 Maintain Shopping Cart

UC-06 Checkout and Submit Order

Table 2.8 describes how a customer completes checkout by providing delivery details and confirming an order. The system validates fields, rechecks stock, creates the order, and updates stockQuantity.

Use Case ID	**UC-06**
Use Case Name	Checkout and Submit Order
Primary Actor	Customer
Stakeholders & Interests	**Customer:** place order successfully. **System Owner:** consistent order record and correct stock update.
Preconditions	1. Customer is logged in. 2. Cart is not empty. 3. Internet connectivity available.
Postconditions	1. Order is created with a unique order ID and recorded persistently. 2. stockQuantity reduced for ordered products. 3. Cart cleared.
Main Success Scenario	1. Customer opens cart and selects Checkout. 2. System displays checkout form. 3. Customer enters delivery details and selects payment option. 4. Customer confirms order. 5. System re-validates stock for all items. 6. System creates order and updates inventory. 7. System shows order confirmation with order ID.
Extensions/Alternative Flows	3A Invalid fields: System shows validation errors and blocks submission. 3B Online payment selected: System initiates payment with the payment gateway and receives success/failure response. 3B1 Payment failed/declined: System shows failure message; order is not created. 3B2 Gateway unavailable: System shows error and offers retry (or allows COD if supported). 5A Stock insufficient: System stops and asks customer to update cart. 6A Write/update failure: System shows error and does not confirm order.
Frequency of Occurrence	Medium.
Special Requirements	Order creation and stock update must be consistent (no partial confirmation).
Technology	Firestore (Orders); atomic update strategy at implementation level.
Inputs	Delivery details; payment option; confirm action.
Outputs	1. Order confirmation. 2. Stored order.

	3. Updated stock.

Table 2.8: UC-6 Checkout and Submit Order

UC-07 Review Order History

Table 2.9 describes how a customer views previous orders and their current status. The system retrieves only the customer's orders and displays them.

Use Case ID	**UC-07**
Use Case Name	Review Order History
Primary Actor	Customer
Stakeholders & Interests	**Customer:** view previous purchases and status. **System Owner:** enforce access control per user.
Preconditions	Customer is logged in.
Postconditions	Customer order list (and selected order details) displayed.
Main Success Scenario	1. Customer opens Order History. 2. System retrieves the customers orders. 3. System displays list with status. 4. Customer selects an order to view details.
Extensions/Alternative Flows	2A No orders: System shows empty state. 2B Network failure: System shows error and retry.
Frequency of Occurrence	Medium.
Special Requirements	Customer must not view other users orders (security rules).
Technology	Firestore query by userId.
Inputs	Order history request; order selection.
Outputs	Order list and order details displayed.

Table 2.9: UC-7 Review Order History

UC-08 Maintain Customer Profile

Table 2.10 describes how a customer views or updates profile information used for contact and delivery purposes. The system validates and saves the updated profile information.

Use Case ID	**UC-08**
Use Case Name	Maintain Customer Profile
Primary Actor	Customer
Stakeholders & Interests	**Customer:** keep contact info accurate. **System Owner:** maintain valid profile data for orders.
Preconditions	Customer is logged in.
Postconditions	Profile information is updated.
Main Success Scenario	1. Customer opens Profile screen. 2. System loads profile data. 3. Customer edits fields (e.g., name/phone). 4. Customer selects Save. 5. System validates and saves the profile updates.

Extensions/Alternative Flows	4A Invalid data: System shows validation error; update not saved. 5A Network failure: System shows error and retry.
Frequency of Occurrence	Low to Medium.
Special Requirements	Required fields must not be empty; validation must be clear.
Technology	Firestore (User Profile).
Inputs	Edited profile fields; save action.
Outputs	Updated profile confirmation.

Table 2.10: UC-8 Maintain Customer Profile

UC-09 Maintain Product Catalog

Table 2.11 describes how an administrator adds, edits, or deletes product records. The system validates fields and saves the changes.

Use Case ID	**UC-09**
Use Case Name	Maintain Product Catalog
Primary Actor	Administrator
Stakeholders & Interests	**Admin:** manage products efficiently. **Customers:** see accurate products and pricing.
Preconditions	1. Admin is logged in via admin interface. 2. Admin has required privileges.
Postconditions	Product record created/updated/deleted.
Main Success Scenario	1. Admin opens Product Management. 2. Admin selects add/edit/delete. 3. Admin enters/updates product fields. 4. Admin submits changes. 5. System validates and saves the product changes. 6. System confirms success.
Extensions/Alternative Flows	3A Missing required data: System blocks and prompts correction. 5A Database failure: System shows error and allows retry.
Frequency of Occurrence	Medium.
Special Requirements	Only authorized admin can modify product catalog.
Technology	Admin interface; Firestore (Products).
Inputs	Product fields; add/edit/delete action.
Outputs	1. Updated product catalog. 2. Confirmation/error.

Table 2.11: UC-9 Maintain Product Catalog

UC-10 Maintain Product Media Assets

Table 2.12 describes how an administrator uploads product images and AR 3D models. The system uploads media assets and associates them with the selected product.

Use Case ID	**UC-10**
Use Case Name	Maintain Product Media Assets
Primary Actor	Administrator

Stakeholders & Interests	**Admin:** upload correct media for products. **Customers:** view correct images and AR availability.
Preconditions	1. Admin is logged in. 2. Product exists. 3. Internet connectivity available.
Postconditions	1. Media uploaded to Storage. 2. Product updated with image URLs and/or model reference.
Main Success Scenario	1. Admin selects a product. 2. Admin chooses image/model upload. 3. System uploads file(s) to Storage. 4. System associates the uploaded media with the selected product. 5. System confirms success.
Extensions/Alternative Flows	3A Upload fails: System shows error and retry. 3B Unsupported type/size: System blocks upload and shows constraints.
Frequency of Occurrence	Medium.
Special Requirements	Model reference must be valid before enabling AR for that product.
Technology	Firebase Storage; Firestore product update.
Inputs	Selected media files; upload action.
Outputs	1. Media stored. 2. Updated product media references. 3. Confirmation/error.

Table 2.12: UC-10 Maintain Product Media Assets

UC-11 Maintain Inventory/Stock

Table 2.13 describes how an administrator updates product stockQuantity. The system validates non-negative stock values and saves the update.

Use Case ID	**UC-11**
Use Case Name	Maintain Inventory/Stock
Primary Actor	Administrator
Stakeholders & Interests	**Admin:** keep stock accurate. **Customers:** avoid ordering out-of-stock items.
Preconditions	1. Admin is logged in. 2. Product exists.
Postconditions	stockQuantity and/or availability status updated.
Main Success Scenario	1. Admin opens inventory/stock view. 2. Admin selects product. 3. Admin enters new stockQuantity. 4. Admin saves. 5. System validates the stock value and saves the update.
Extensions/Alternative Flows	3A Invalid value: System blocks update and shows message. 5A Database failure: System shows error and retry.
Frequency of Occurrence	Medium.

Special Requirements	stockQuantity must never be negative; admin-only access enforced.
Technology	Firestore (Products stockQuantity).
Inputs	Product selection, stockQuantity value, save action.
Outputs	1. Updated stock. 2. Confirmation/error.

Table 2.13: UC-11 Maintain Inventory/Stock

UC-12 Process Orders

Table 2.14 describes how an administrator reviews customer orders and updates order status. The system saves the updated status and confirms the change.

Use Case ID	**UC-12**
Use Case Name	Process Orders
Primary Actor	Administrator
Stakeholders & Interests	**Admin:** process fulfillment steps. **Customers:** receive correct order status.
Preconditions	1. Admin is logged in. 2. Orders exist (or system supports empty state).
Postconditions	Selected order status updated.
Main Success Scenario	1. Admin opens order management. 2. System lists orders with current status. 3. Admin selects an order. 4. Admin chooses new status. 5. System saves the updated order status. 6. System confirms update.
Extensions/Alternative Flows	2A No orders: System shows empty state. 5A Update fails: System shows error; status remains unchanged.
Frequency of Occurrence	Medium to High.
Special Requirements	Only admin accounts can update status; status must reflect correctly in customer order history.
Technology	Firestore (Orders); admin interface.
Inputs	Order selection; new status value; update action.
Outputs	1. Updated order status. 2. Confirmation/error.

Table 2.14: UC-12 Process Orders

2.8 SYSTEM SEQUENCE DIAGRAMS

System Sequence Diagrams (SSDs) represent the black-box interaction between an external actor and the System for a specific use case scenario. An SSD captures the sequence of system events (requests sent to the system) and the system's responses, without modeling internal components, database calls, or implementation logic. The SSDs below are derived from the main success scenario of each use case, with common alternative outcomes modeled using alt fragments.

2.8.1 SSD-1: Create Customer Account

Figure 2.3 shows the customer registration flow at the system level. The customer submits name, email, and password to the system. If the details are valid and the email is not registered, the system creates the account and returns a customer ID; otherwise, it rejects the request and returns an error message.

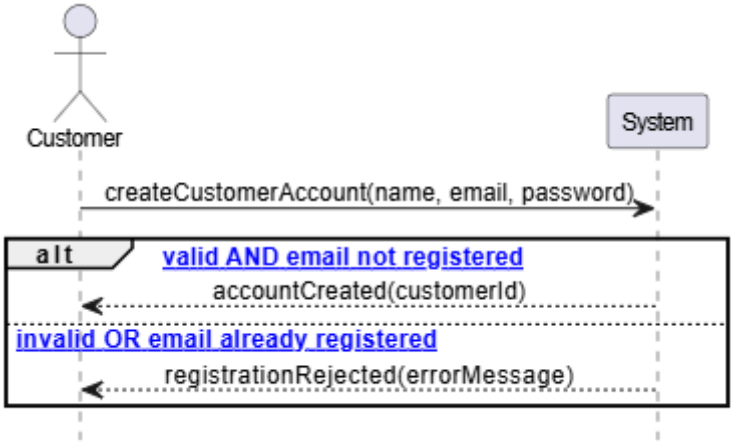

Figure 2.3: SSD-1 Create Customer Account

2.8.2 SSD-2: Explore Product Catalog

Figure 2.4 shows how the customer requests the product catalog from the system. The system returns the product list and, when the customer applies search keywords or category filters, the system returns the updated filtered results or an empty list if no matches are found.

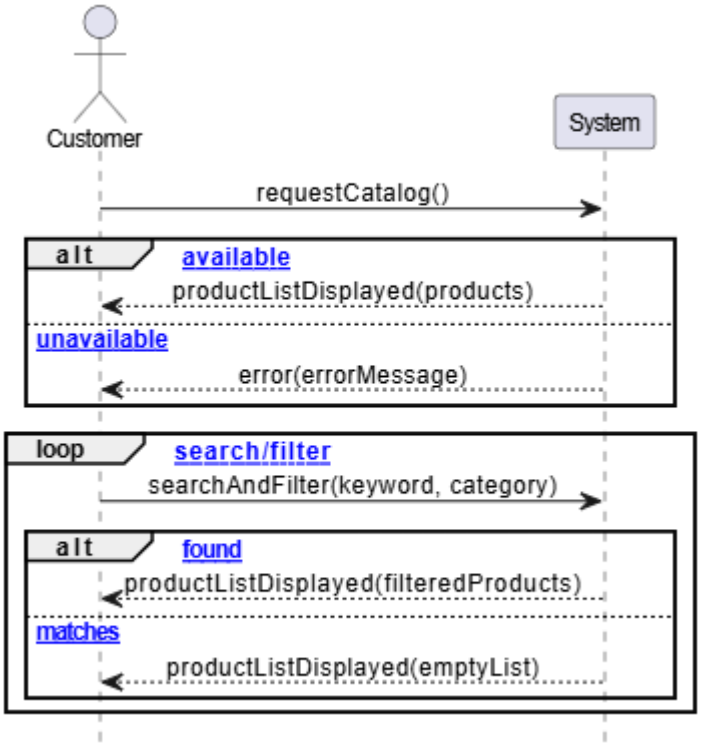

Figure 2.4: SSD-2 Explore Product Catalog

2.8.3 SSD-3: Review Product Details

Figure 2.5 shows that the customer selects a product and requests details from the system. The system responds by displaying product information along with stock status and AR availability or returns an error if the product is unavailable.

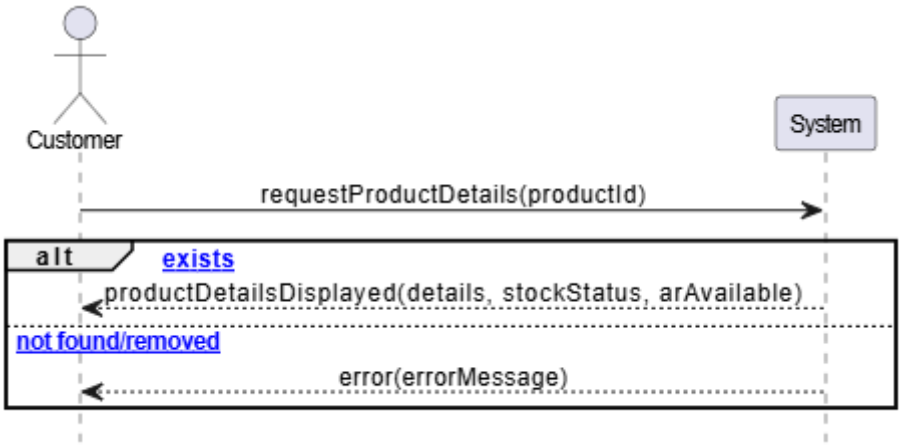

Figure 2.5: SSD-3 Review Product Details

2.8.4 SSD-4: Visualize Product in Augmented Reality

Figure 2.6 shows the customer requesting AR preview for a product. If AR is supported and a 3D model is available, the system requests camera permission and launches the AR preview, otherwise, the system rejects the request with a clear message (e.g., permission denied, unsupported device, or missing model).

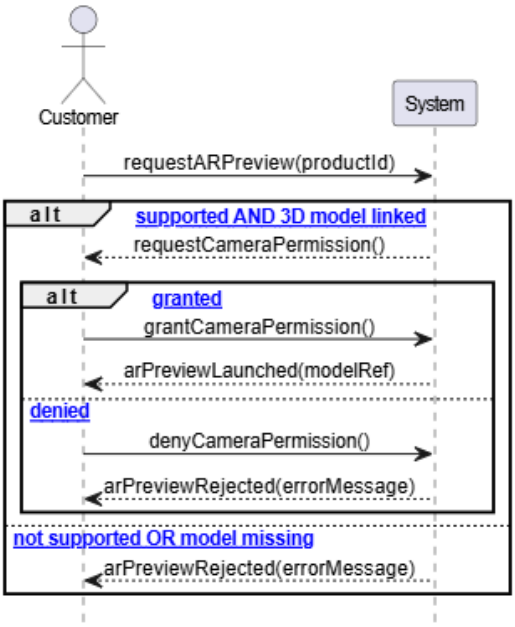

Figure 2.6: SSD-4 Visualize Product in AR

2.8.5 SSD-5: Maintain Shopping Cart

Figure 2.7 shows how the customer adds items to the cart and optionally updates quantities or removes items. The system validates requested quantities against available stock and returns an updated cart summary or rejects the action if stock limits are exceeded.

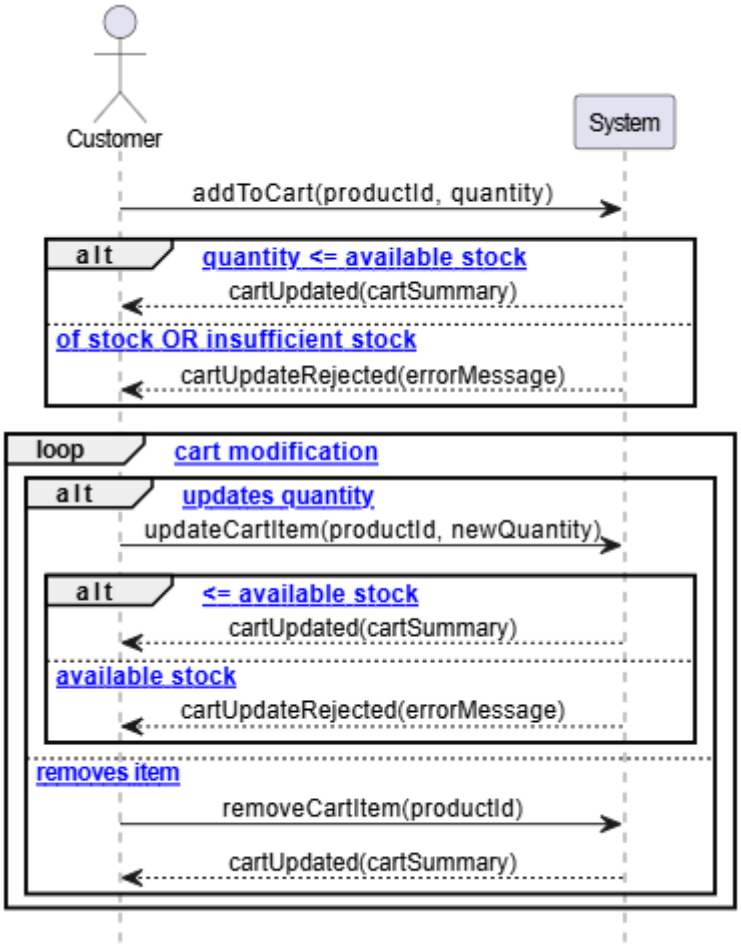

Figure 2.7: SSD-5 Maintain Shopping Cart

2.8.6 SSD-6: Checkout and Submit Order

Figure 2.8 shows the customer submitting delivery details and a payment option to place an order. The system validates the input and verifies stock availability, then confirms the order with an order ID or rejects the request if validation fails or stock is insufficient.

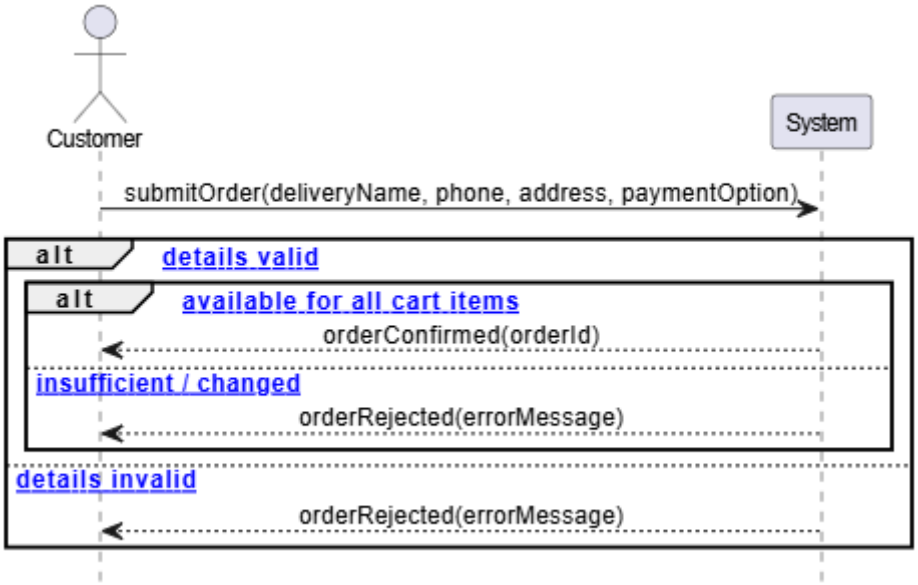

Figure 2.8: SSD-6 Checkout and Submit Order

2.8.7 SSD-7: Review Order History

Figure 2.9 shows the customer requesting order history from the system. The system returns the customer's orders (or an empty list if none exist), and the customer can request details of a specific order to view items and current status.

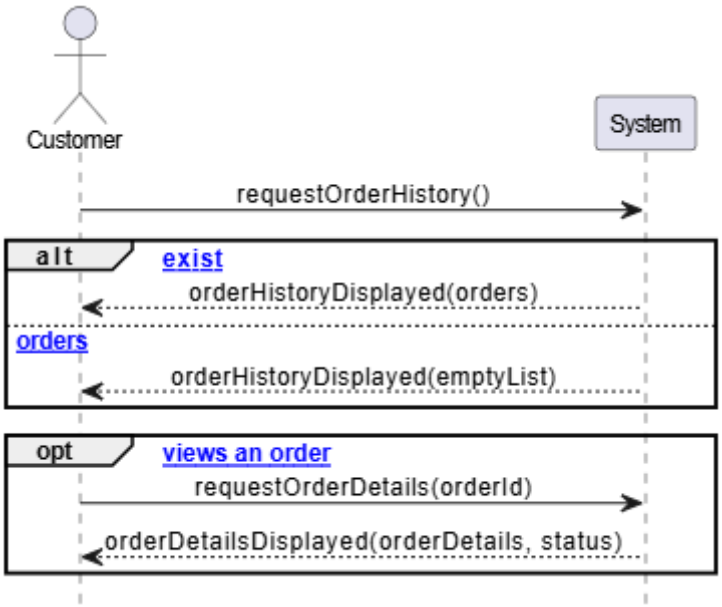

Figure 2.9: SSD-7 Review Order History

2.8.8 SSD-8: Maintain Customer Profile

Figure 2.10 shows the customer updating profile information through the system. The system validates the updated fields and either confirms the profile update or rejects it with an error message.

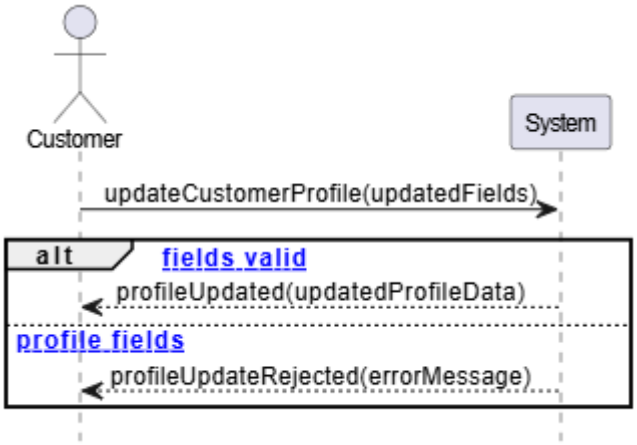

Figure 2.10: SSD-8 Maintain Customer Profile

2.8.9 SSD-9: Maintain Product Catalog

Figure 2.11 shows the administrator performing product catalog maintenance by adding, updating, or deleting products. The system validates the request and confirms success or returns an error if the operation fails.

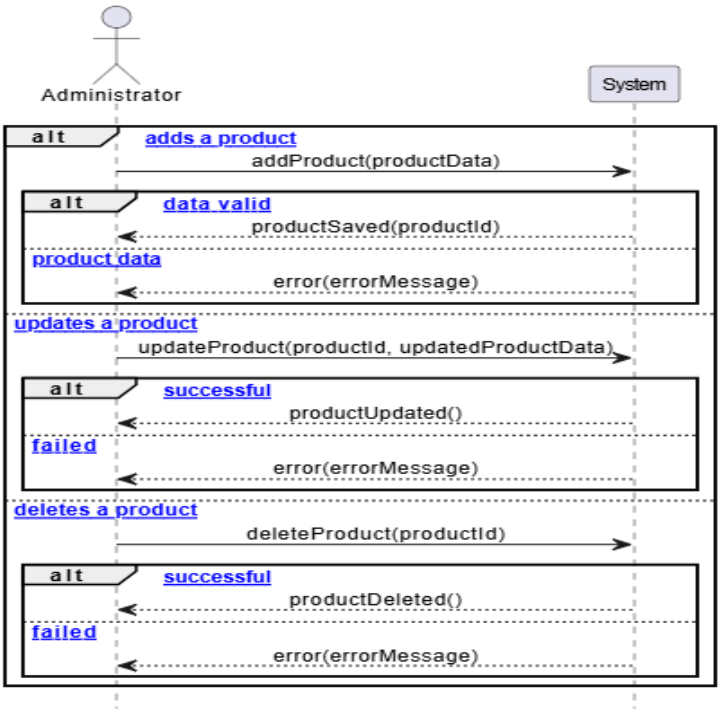

Figure 2.11: SSD-9 Maintain Product Catalog

2.8.10 SSD-10: Maintain Product Media Assets

Figure 2.12 shows the administrator uploading product images or associating a 3D model for AR preview. The system stores the media and confirms the generated URLs/model reference or returns an error if the upload or validation fails.

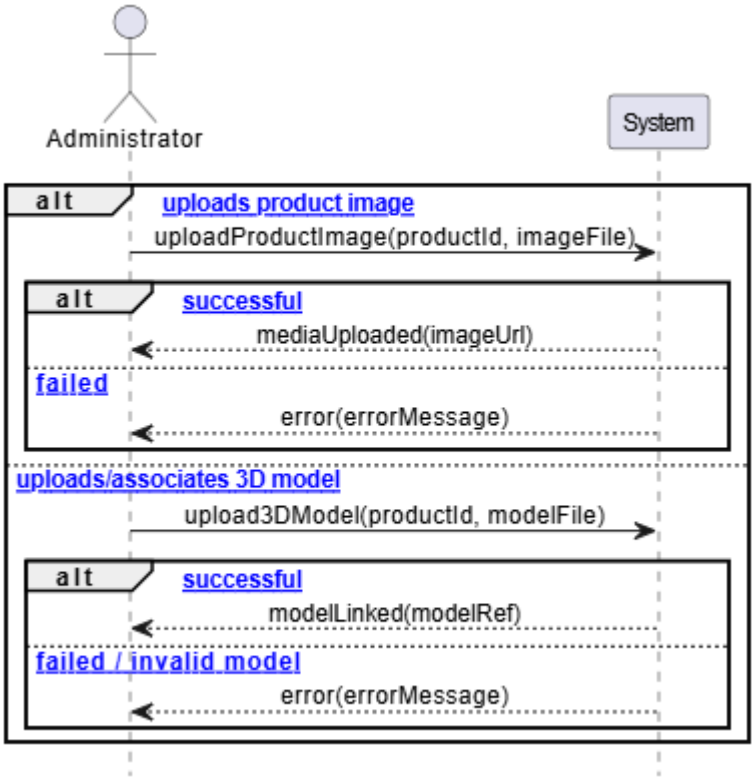

Figure 2.12: SSD-10 Maintain Product Media Assets

2.8.11 SSD-11: Maintain Inventory/Stock

Figure 2.13 shows the administrator updating stock quantity for a product. The system validates that the value is a non-negative integer and confirms the update, or rejects it if the value is invalid.

Figure 2.13: SSD-11 Maintain Inventory/Stock

2.8.12 SSD-12: Process Orders

Figure 2.14 shows the administrator updating the status of an order (e.g., pending to shipped). The system saves the updated status and confirms success or returns an error if the update fails.

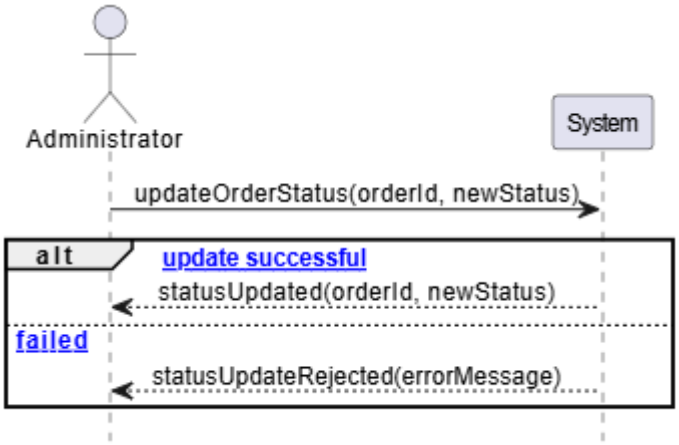

Figure 2.14: SSD-12 Process Orders

2.9 DOMAIN MODEL

A domain model is a conceptual view of the problem domain that identifies the system's core real-world entities, their key attributes, and the associations between them. It helps define what information the system must manage.

Figure 2.15 shows the conceptual domain entities of the AR-enabled e-commerce system and their relationships. It models how a customer maintains a cart and places orders containing order items, while each order includes delivery details and optional payment information. The product catalog is represented through categories, product images, optional AR 3D model

assets, and inventory stock. The model also includes enumerations (OrderStatus, PaymentMethod, PaymentStatus) to define valid states for order processing and payments.

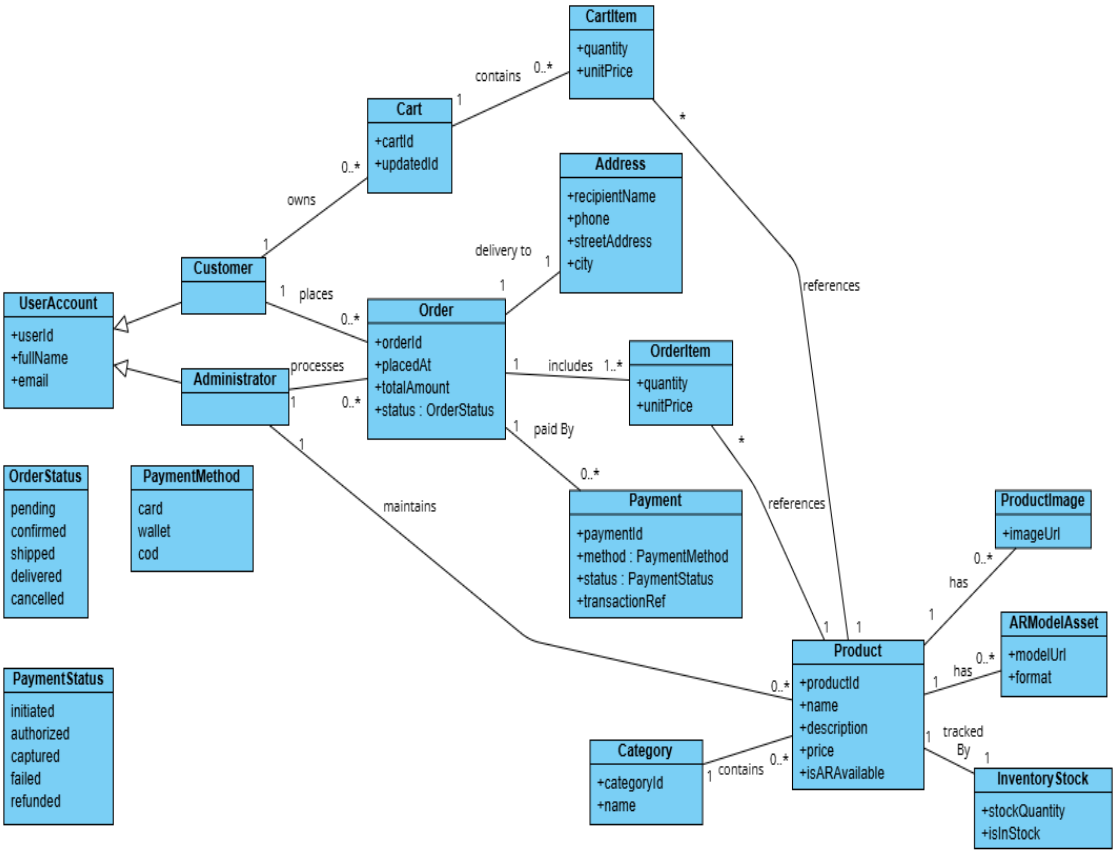

Figure 2.15: Domain Model

CHAPTER # 3

SOFTWARE DESIGN DESCRIPTION (SDD)

3.1 INTRODUCTION

This chapter presents the Software Design Description (SDD) for the E-Commerce Platform with Augmented Reality. It summarizes the system's design approach and major modules that support product browsing, AR preview, cart and checkout, and order management. The design is modular to keep components loosely coupled and easier to implement, test, and maintain.

3.1.1 Design Overview

A layered, modular design is used for the AR-enabled e-commerce system. Customers can browse products, view details, preview items in AR (when a 3D model is available), manage the cart, and place orders through checkout. Administrators manage the product catalog, media assets, stock, and order status updates. This separation keeps the system maintainable and supports incremental development and testing.

Major elements include:

- **Customer Application (Presentation Layer)**
  Provides interfaces for account creation/login, catalog browsing, search/filter, product details, AR preview, cart operations, checkout, order history, and profile management.
- **Administrator Module (Administration Layer)**
  Provides interfaces for maintaining the product catalog, uploading/associating product media (images and AR 3D models), updating inventory values, and managing order status.
- **Application Logic Layer (Core Services)**
  Implements the main business rules such as validation of user input, cart total calculation, stock checks during cart/checkout, order creation, and order status updates.
- **Data Management Layer (Repositories / Storage Access)**
  Handles persistent storage of structured records (users, products, carts, orders, inventory) and retrieval of media assets (product images and 3D models).
- **AR Preview Subsystem**
  Manages AR session initialization, permission handling, plane detection, and rendering of 3D product models in the customer's environment.
- **Payment Handling Module**
  Supports online payment through a payment gateway when integrated. Otherwise, the same checkout workflow can proceed using COD or simulated payment confirmation in the prototype.

3.1.2 Requirements Traceability Matrix

Table 3.1 presents the Requirements Traceability Matrix (RTM). It links each requirement/use case to the interface or module that implements it and the test case used for verification. This mapping helps ensure that all requirements are covered in the design and can be tested and validated.

Requirement ID	Requirement Name	Interface/Module	Test Case
UC-01	Create Customer Account	Signup/Login + Authentication Module	TC-01: Registration Validation (valid/duplicate/invalid)
UC-02	Explore Product Catalog	Catalog UI + Catalog Management Module	TC-02: Catalog Load + Search/Filter Accuracy
UC-03	Review Product Details	Product Details UI + Product Information Module	TC-03: Product Detail Integrity (price/media/stock)
UC-04	Visualize Product in Augmented Reality	AR Preview UI + AR Preview Subsystem + Media Access	TC-04: AR Launch (permission/device/model cases)
UC-05	Maintain Shopping Cart	Cart UI + Cart Management Module	TC-05: Cart Operations (add/update/remove + totals)
UC-06	Checkout and Submit Order	Checkout UI + Order Management Module	TC-06: Order Creation + Stock Recheck + Confirmation
UC-07	Review Order History	Order History UI + Order Query Module	TC-07: Order History Access Control + Listing
UC-08	Maintain Customer Profile	Profile UI + User Profile Module	TC-08: Profile Update Validation + Persistence
UC-09	Maintain Product Catalog	Admin UI + Product Administration Module	TC-09: Product CRUD + Field Validation
UC-10	Maintain Product Media Assets	Admin UI + Media Asset Management Module	TC-10: Image/Model Upload + Product Linking
UC-11	Maintain Inventory/Stock	Admin UI + Inventory Management Module	TC-11: Stock Update Rules (non-negative + reflected)
UC-12	Process Orders	Admin UI + Order Processing Module	TC-12: Order Status Update + Customer Visibility

Table 3.1: Requirements Traceability Matrix

3.2 SYSTEM ARCHITECTURAL DESIGN

3.2.1 Chosen System Architecture

A Three-Tier (Layered) Architecture is selected for this project because it provides a clear separation between user interaction, application logic, and data storage. It also fits naturally with Firebase-based systems and keeps the design modular and easy to maintain.

1. **Presentation Layer (Client Layer)**
   - **Technology:** Flutter (Android / iOS)
   - **Responsibilities:**

- **Customer Module:** registration/login, product browsing/search/filter, product details, AR preview, cart management, checkout, order history, and profile management.
- **Admin Module (In-App):** product management, media upload/association (images and 3D models), inventory/stock updates, and order processing (status updates).
- Handles user input, navigation flow, and screen-level validations.

2. **Application Layer (Business Logic Layer)**
   - **Technology:** Flutter service modules (app-side logic)
   - **Responsibilities:**
     - Apply business rules such as form validation, cart total calculation, and checkout validation.
     - Verifies stock availability at checkout time.
     - Creates orders and updates order status based on the defined workflow.
     - Ensures data consistency during order placement by using Firestore Transactions so that stock validation and stock decrement occur atomically.
3. **Data Layer (Persistent Storage Layer)**
   - **Technology:** Firebase Cloud Firestore + Firebase Storage
   - **Responsibilities:**
     - **Cloud Firestore:** stores users, products, categories, carts, orders, and inventory values (stockQuantity)**.**
     - **Firebase Storage:** stores product images and AR 3D model assets linked to products.
     - Uses access control through Firebase Security Rules to restrict admin-only operations (catalog/inventory/order management) and protect user-specific data (carts/orders).

3.2.2 Discussion of Alternative Designs

- **Two-Tier Architecture**
  In a two-tier approach, most logic is handled directly in the client while communicating with the database/storage layer in a tightly coupled way. It was not selected because:
  - It increases complexity in enforcing consistent business rules, especially for inventory updates and order consistency.
  - As features grow (AR + commerce + admin), the client becomes harder to maintain without clear layering.
  - Security and access control become more difficult to manage cleanly.
- **Monolithic Custom Backend Architecture**
  A monolithic backend (custom server + database) was also considered. It was not selected because:
  - It increases hosting and maintenance overhead (API development, deployment, scaling).
  - Firebase already provides authentication, structured storage, and media storage required for this project.
  - The project scope is better served by focusing on core features (AR preview, shopping flow, admin management) rather than building server infrastructure.

3.2.3 System Interface Description

The system integrates through the following interfaces:

Customer Interface (Mobile UI)

- Allows customers to register/login, browse products, view product details, preview items in AR, manage cart, checkout, and view order history/profile.
- Supports interaction through forms, buttons, lists, and confirmation screens.

Admin Interface (In-App Admin Module)

- Allows administrators to add/update/delete products, upload/associate images and 3D models, update stockQuantity, and update order status.
- Access is restricted to authorized admin accounts.

Authentication Interface

- Manages account registration and login sessions.
- Ensures protected operations require authentication, and admin actions require admin authorization.

Database Interface (Cloud Firestore Access)

- Stores and retrieves structured data including products, categories, carts, orders, user profiles, inventory, and order statuses.
- Supports query-based access for catalog listing, filtering, and customer-specific order history.

Media Storage Interface (Firebase Storage Access)

- Stores product images and AR 3D assets.
- Provides secure references/URLs used by the application to load images and render AR models.

AR Framework Interface

- Creates AR sessions, requests camera permission, detects surfaces, and renders the selected product's 3D model.
- Handles AR failure cases such as unsupported device, permission denial, and tracking/lighting limitations.

Payment Gateway Interface

- Supports online payment authorization/capture when implemented.
- Returns payment success/failure status to complete or reject the checkout flow accordingly.

3.3 DETAILED DESCRIPTION OF COMPONENTS

This section describes the key components of the E-Commerce Platform with Augmented Reality based on the selected layered architecture. The system is organized into presentation, application logic, and data/storage responsibilities. Each component is designed to fulfill a

specific set of requirements and works with the others to support product exploration, AR preview, cart and checkout, inventory handling, and order processing.

3.3.1 User Interface (Presentation Layer)

The user interface manages all user-system interaction through the mobile application. It provides the screens and flows required for both roles: Customer and Administrator. For customers, it includes signup/login, catalog browsing, product details, AR preview, cart, checkout, order history, and profile screens. For administrators, it includes product management, media upload/association, inventory update, and order status update screens. The UI ensures consistent navigation, input validation at form level, and clear feedback for success/failure states.

3.3.2 Controller and State Management Component

This component acts as the coordination layer between UI screens and the underlying services. It receives user actions (Add to Cart, Checkout, Upload Model), triggers the required service operations, and updates the UI state accordingly. It is responsible for managing screen transitions, loading/error states, and keeping the UI synchronized with current system data (updated stock status, cart totals, order status changes).

3.3.3 Authentication and Authorization Component

The authentication component handles user registration, login, and session management using the authentication service. The authorization part enforces role-based access so that customer features are restricted to authenticated customers, while administrative features are accessible only to authorized admin accounts. This component also ensures that protected operations (cart, checkout, order history, admin screens) are not accessible without valid authentication.

3.3.4 Catalog and Product Management Component

This component manages the product catalog lifecycle. It retrieves and displays product categories and product lists, supports search/filter operations, and provides detailed product information when selected. It also ensures that key product attributes used by the app are consistently available, such as price, images, stockQuantity, and the AR model reference (when applicable). Administrative product CRUD operations (add/update/delete) are executed through this component with validation of required fields.

3.3.5 AR Preview Component

The AR preview component enables AR-based visualization for supported products and supported devices. It initializes an AR session, handles camera permission checks, loads the linked 3D model asset, and renders the model on detected surfaces. It also provides guided feedback when AR cannot be started (permission denied, unsupported device, model load failure, poor tracking/surface detection), ensuring the user can safely return to product details without application instability.

3.3.6 Cart Management Component

The cart component manages user-specific cart operations including add, remove, and quantity update. It recalculates subtotal/total and enforces stock constraints so that

requested quantities do not exceed stockQuantity. Cart state is stored per user to support continuity across sessions and devices, and it remains consistent with the product catalog (out-of-stock handling and quantity limits).

3.3.7 Checkout, Order Processing, and Payment Component

This component implements the checkout workflow, validates delivery details, and processes payment outcomes when online payment is selected. On order confirmation, it performs atomic order placement using Firestore Transactions to ensure consistency: stock is rechecked, the order is created, stockQuantity is decremented for ordered items, and the cart is cleared as a single consistent operation. It also supports administrative order processing by enabling status transitions (pending to shipped to delivered) and ensuring the updated status is reflected in the customers order history.

3.3.8 Data Access and Media Storage Component (Data Layer)

This component encapsulates all interaction with persistent storage. Structured records (users, products, carts, orders, inventory values) are stored and retrieved from the database service, while product images and 3D model assets are stored in media storage and linked using references/URLs. It also enforces access control through security rules so that customers can only access their own cart/orders, while catalog/inventory/order updates remain restricted to administrators.

3.4 ARCHITECTURAL DIAGRAM

Figure 3.1 shows the three-tier architecture of the AR-enabled e-commerce system. The Presentation Layer contains the Flutter customer and admin interfaces, the Application Layer contains the controller/state management and service modules, and the Data Layer includes Firebase services, ARCore/ARKit, and the external payment gateway.

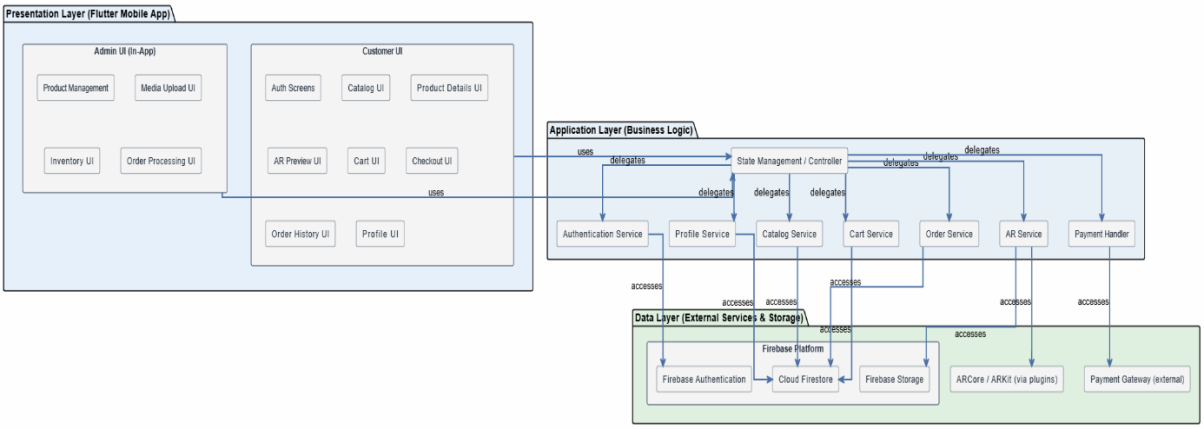

Figure 3.1: Three-Tier Layered Architecture Diagram

3.5 SEQUENCE DIAGRAMS

Sequence diagrams show how actors and internal components exchange messages over time to complete a specific use case. Unlike system sequence diagrams (SSDs), these diagrams expand the internal collaboration between UI screens (boundary), controllers/services (control), domain objects (entity), and external platforms.

3.5.1 SD-1: Create Customer Account

Figure 3.2 shows customer registration flow where the Signup/Login UI sends details to AuthController/AuthService. The system creates the user in Firebase Authentication and stores the profile in Cloud Firestore on success. Otherwise, it returns an error and the UI displays the failure message.

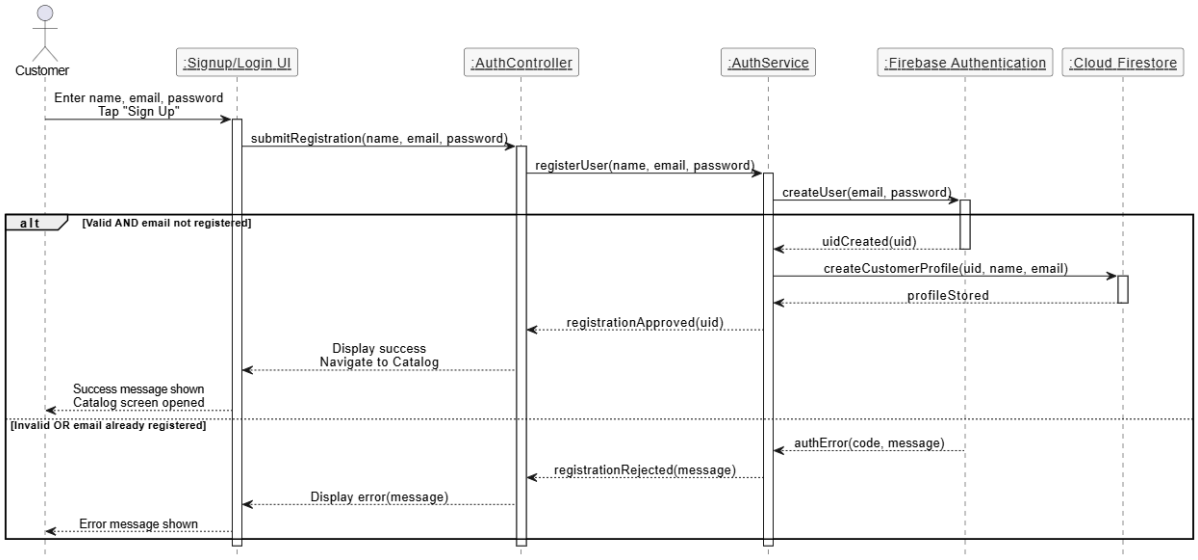

Figure 3.2: SD-01 Create Customer Account

3.5.2 SD-2: Explore Product Catalog

Figure 3.3 shows how the Customer UI requests the product catalog from the Catalog Service, which retrieves product records from Cloud Firestore and returns them for rendering. If the request fails, the UI displays an error and supports retry.

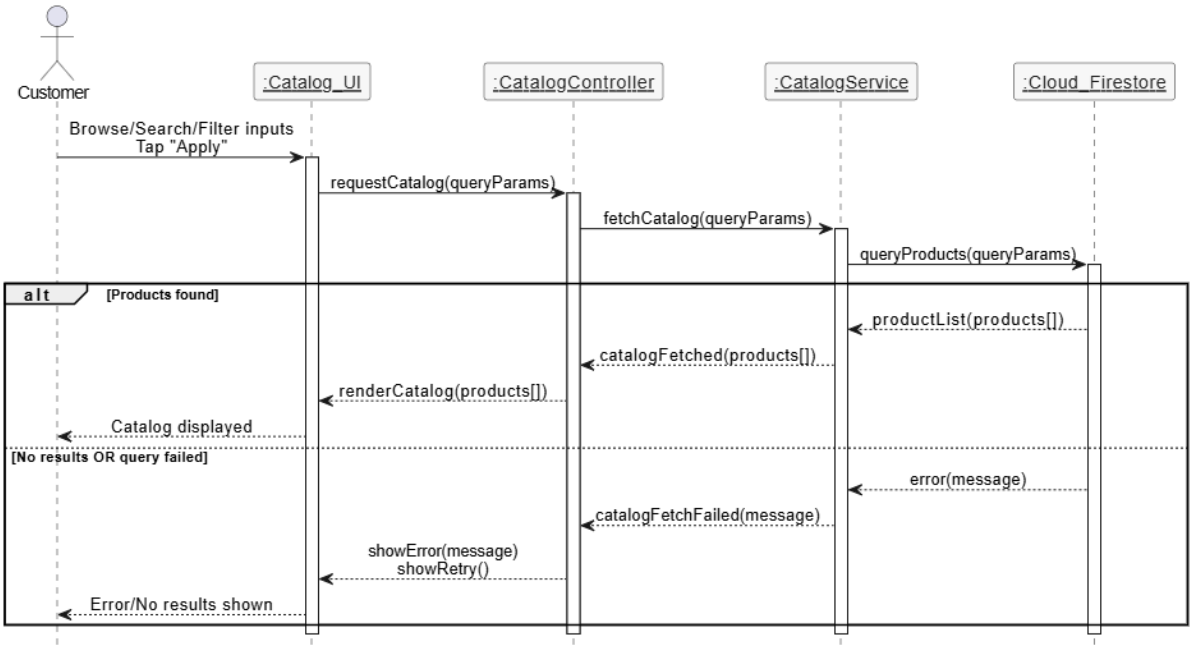

Figure 3.3: SD-02 Explore Product Catalog

3.5.3 SD-3: Review Product Details

Figure 3.4 shows the Product Details UI fetching a selected products full information via the Catalog Service from Cloud Firestore, including price, description, media links, stock indicator, and AR availability.

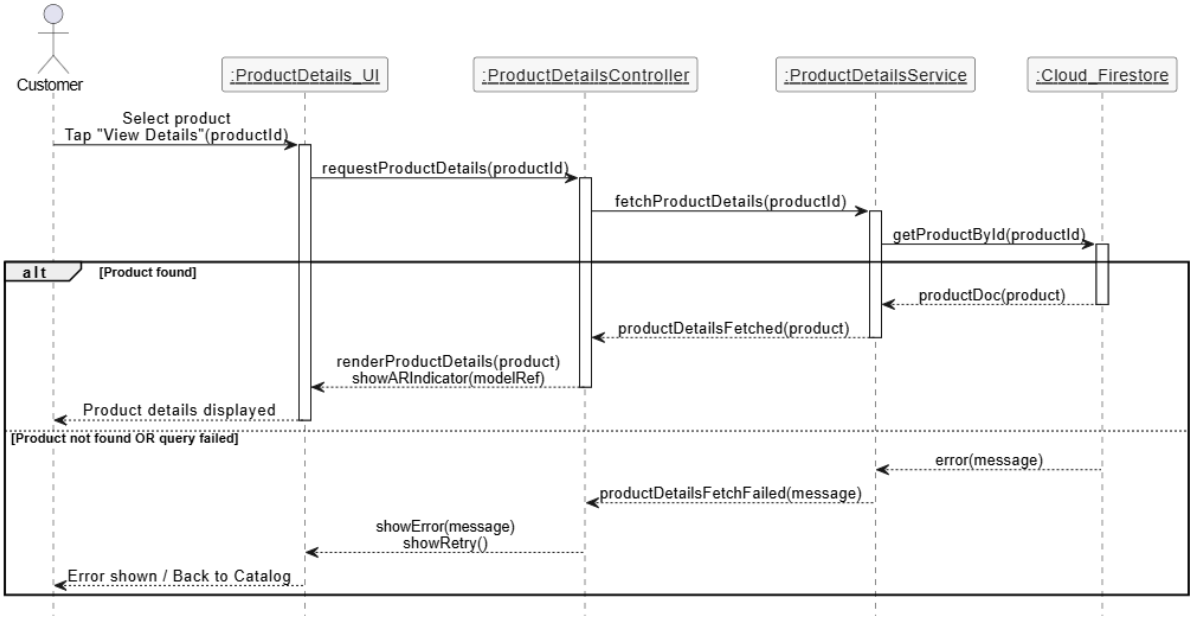

Figure 3.4: SD-03 Review Product Details

3.5.4 SD-4: Visualize Product in Augmented Reality

Figure 3.5 shows the AR Preview UI initiating an AR session through the AR Service, which loads the products 3D model from Firebase Storage and uses ARCore/ARKit to render it. Alternative paths handle missing permission, unsupported device, or missing model.

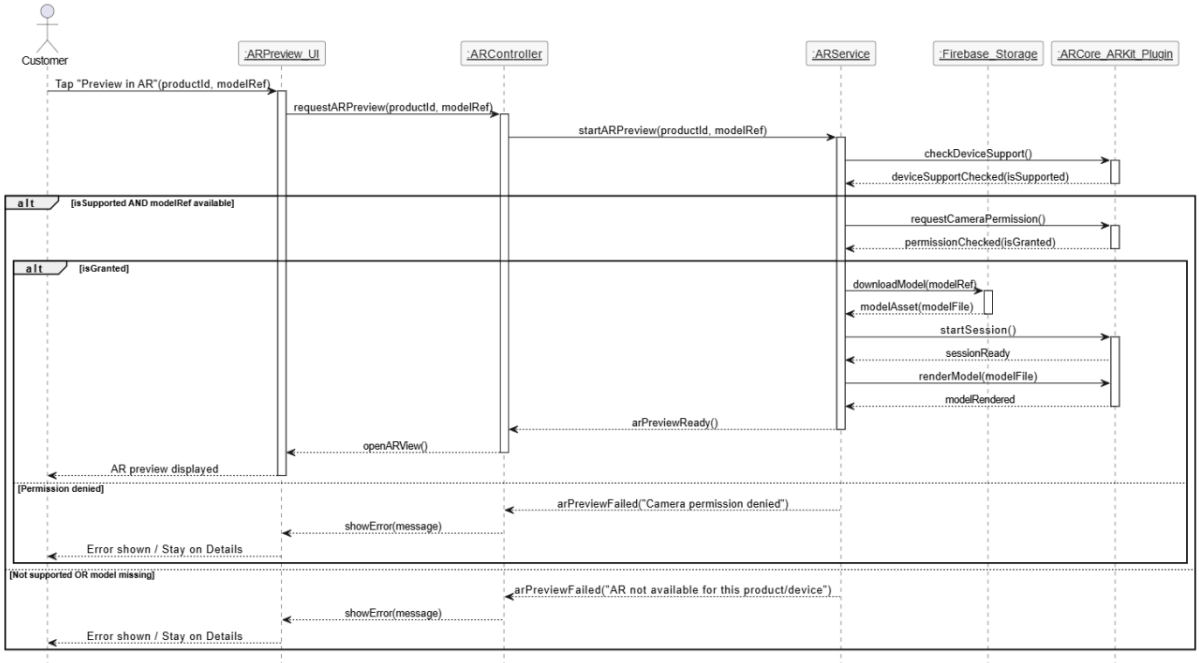

Figure 3.5: SD-04 Visualize Product in AR

3.5.5 SD-5: Maintain Shopping Cart

Figure 3.6 shows cart operations (add/update/remove) issued from the Customer UI to the Cart Service, which updates the user's cart state in Cloud Firestore and returns the refreshed totals to the UI.

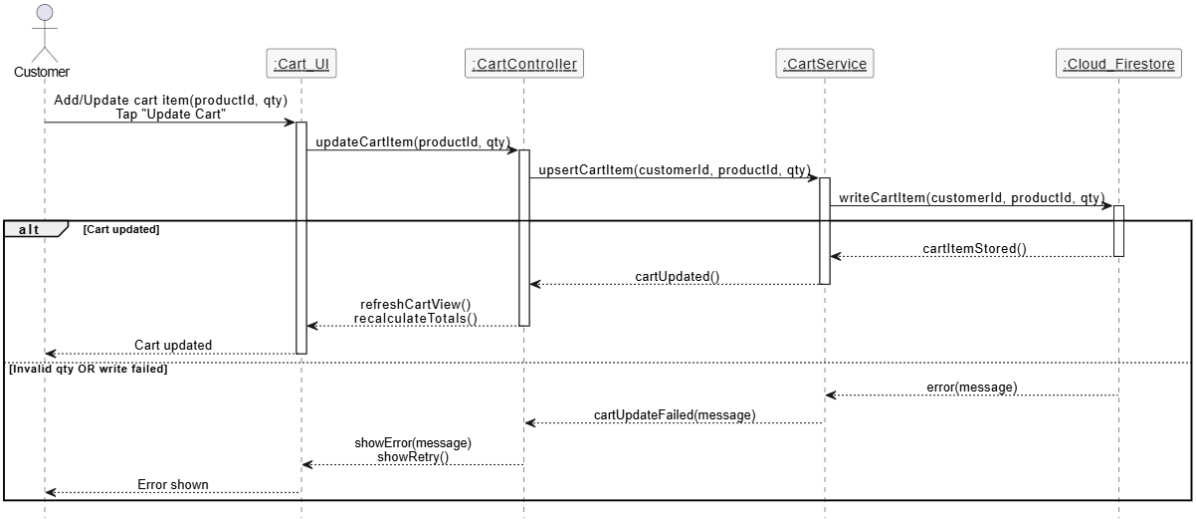

Figure 3.6: SD-05 Maintain Shopping Cart

3.5.6 SD-6: Checkout and Submit Order

Figure 3.7 shows checkout submission where the Checkout UI calls the Order Service to place an order using a Firestore transaction (stock validation, order creation, stock decrement, and cart clearing), and the Payment Handler interacts with the Payment Gateway when online payment is selected.

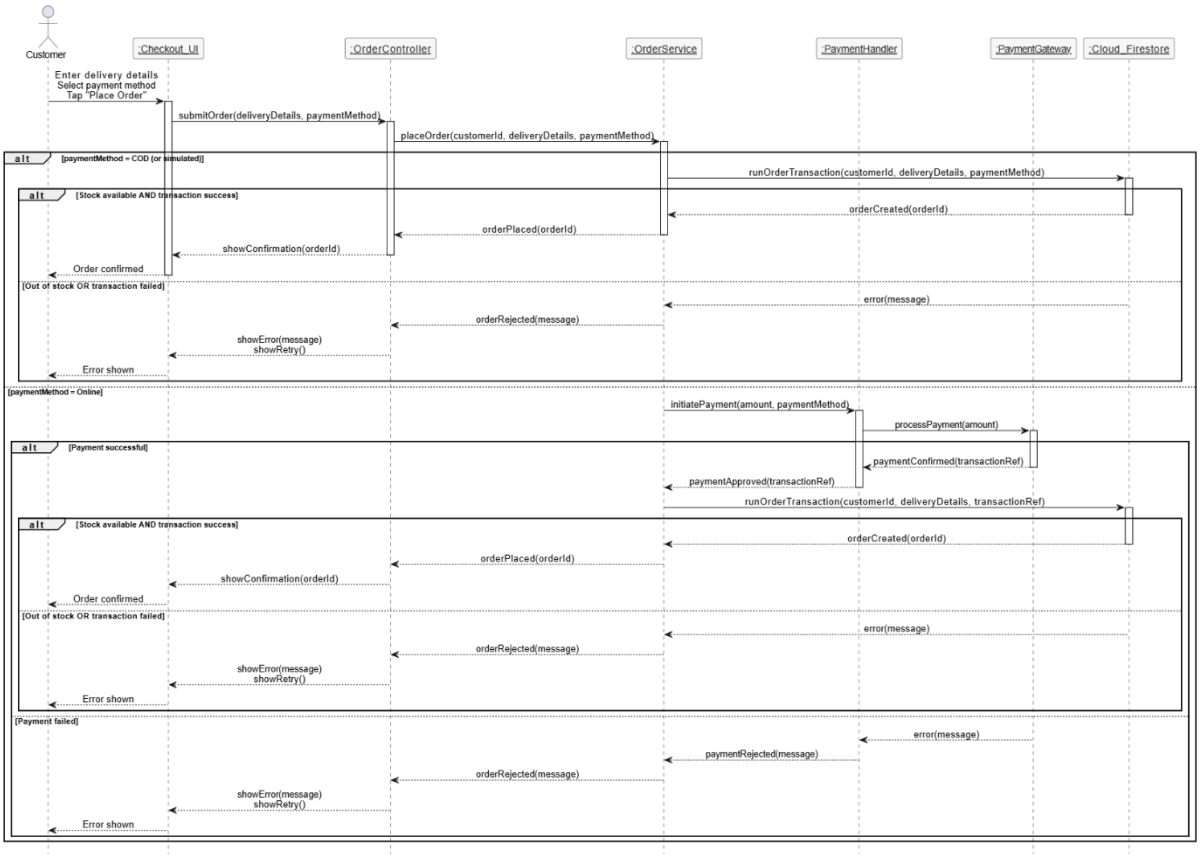

Figure 3.7: SD-06 Checkout and Submit Order

3.5.7 SD-7: Review Order History

Figure 3.8 shows the Order History UI requesting the customers orders from the Order Service, which queries Cloud Firestore using the authenticated user context and returns the list for display.

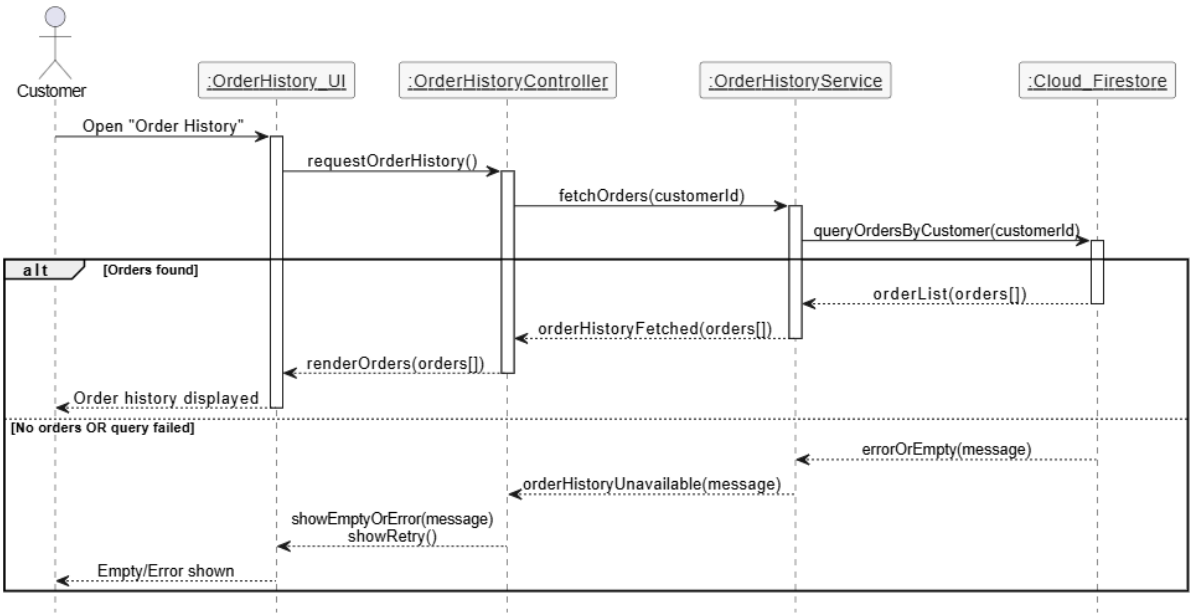

Figure 3.8: SD-07 Review Order History

3.5.8 SD-8: Maintain Customer Profile

Figure 3.9 shows the Profile UI submitting profile updates to the Profile Service, which validates and persists the changes in Cloud Firestore and returns a success/error response to the UI.

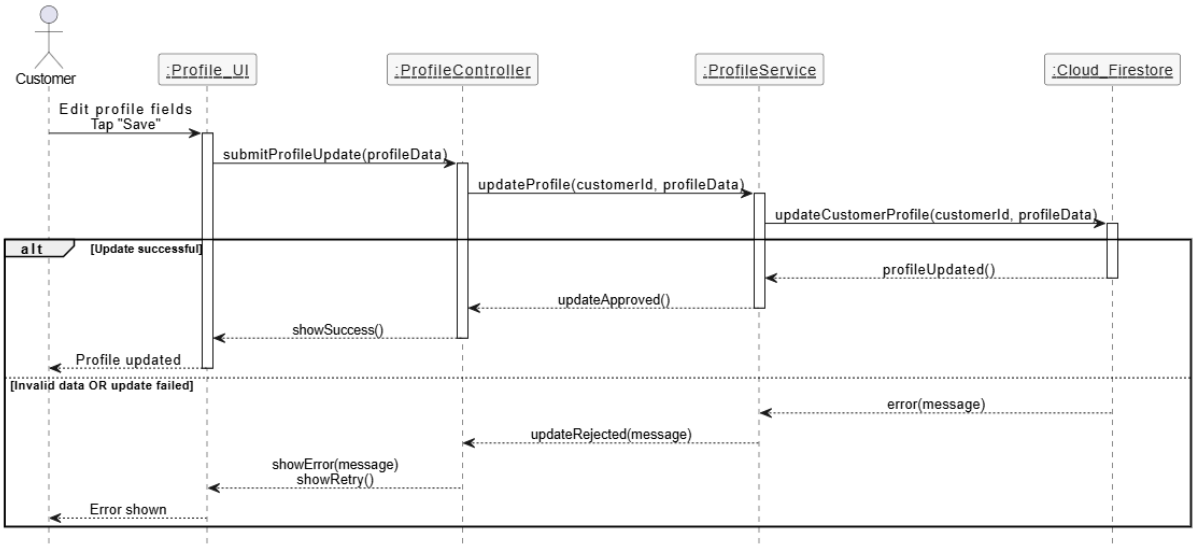

Figure 3.9: SD-08 Maintain Customer Profile

3.5.9 SD-9: Maintain Product Catalog

Figure 3.10 shows the Admin UI performing product create/update/delete operations via the Admin Management Service, which applies changes to product records in Cloud Firestore and returns operation status.

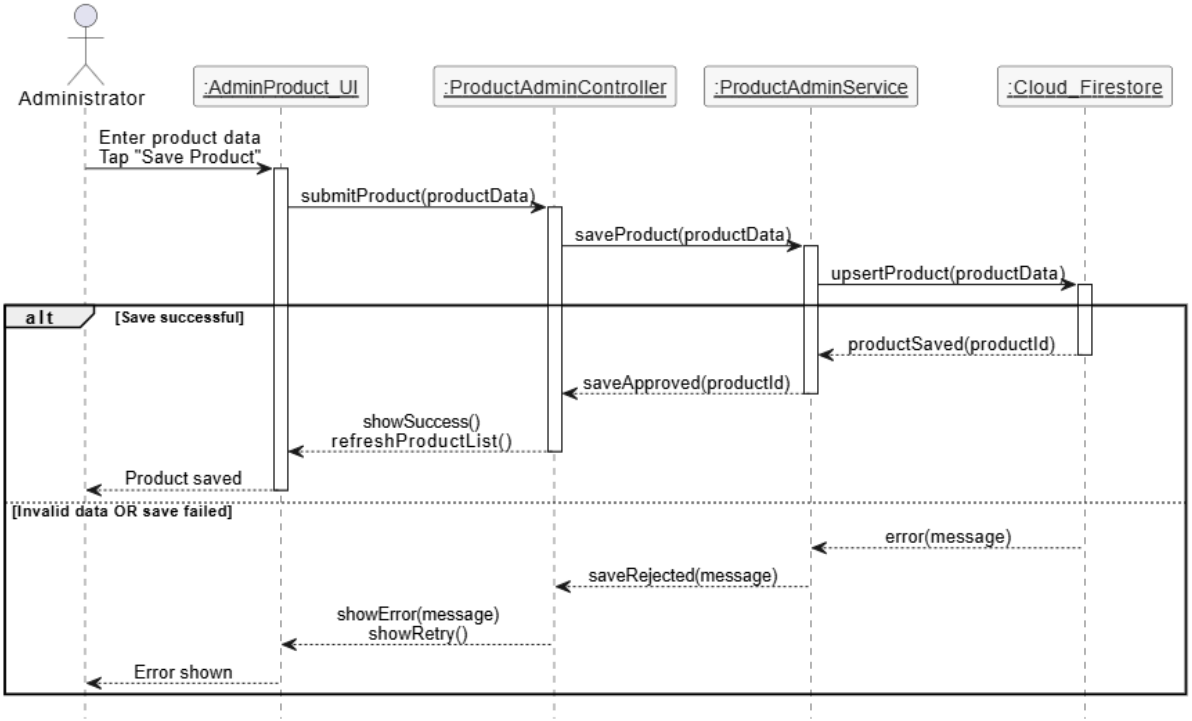

Figure 3.10: SD-09 Maintain Product Catalog

3.5.10 SD-10: Maintain Product Media Assets

Figure 3.11 shows the admin uploading product images/3D models through the Admin UI, where the Media Handler stores files in Firebase Storage, obtains URLs, and updates the related product document in Cloud Firestore.

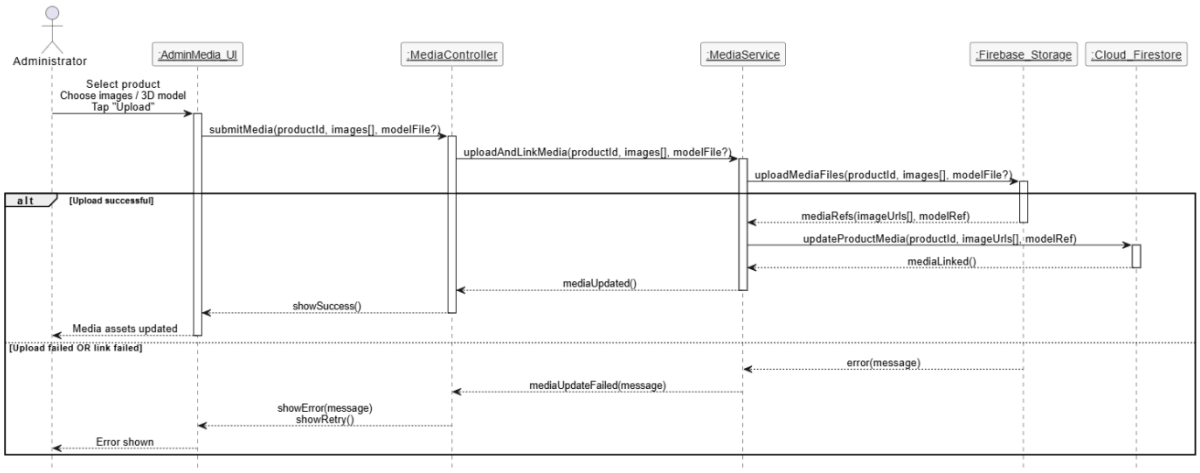

Figure 3.11: SD-10 Maintain Product Media Assets

3.5.11 SD-11: Maintain Inventory/Stock

Figure 3.12 shows the admin updating stock quantities from the Inventory UI through the Admin/Inventory Service, which writes the updated stock fields to Cloud Firestore with validation for non-negative quantities.

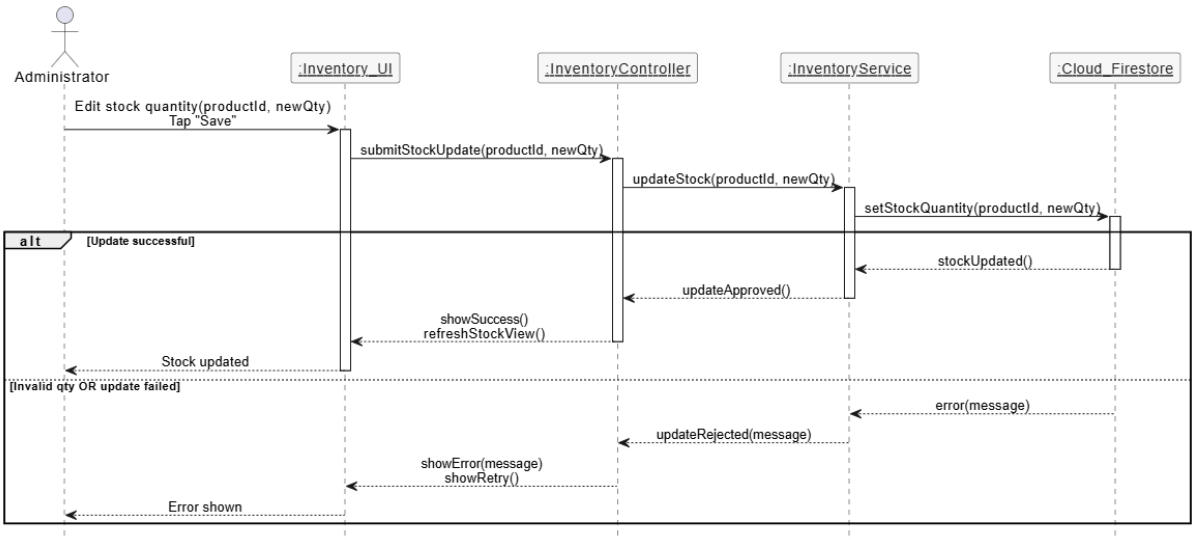

Figure 3.12: SD-11 Maintain Inventory/Stock

3.5.12 SD-12: Process Orders

Figure 3.13 shows the admin retrieving orders and updating order status via the Order Management UI and Order Service, which updates the order status fields in Cloud Firestore so the customer can see the latest tracking state.

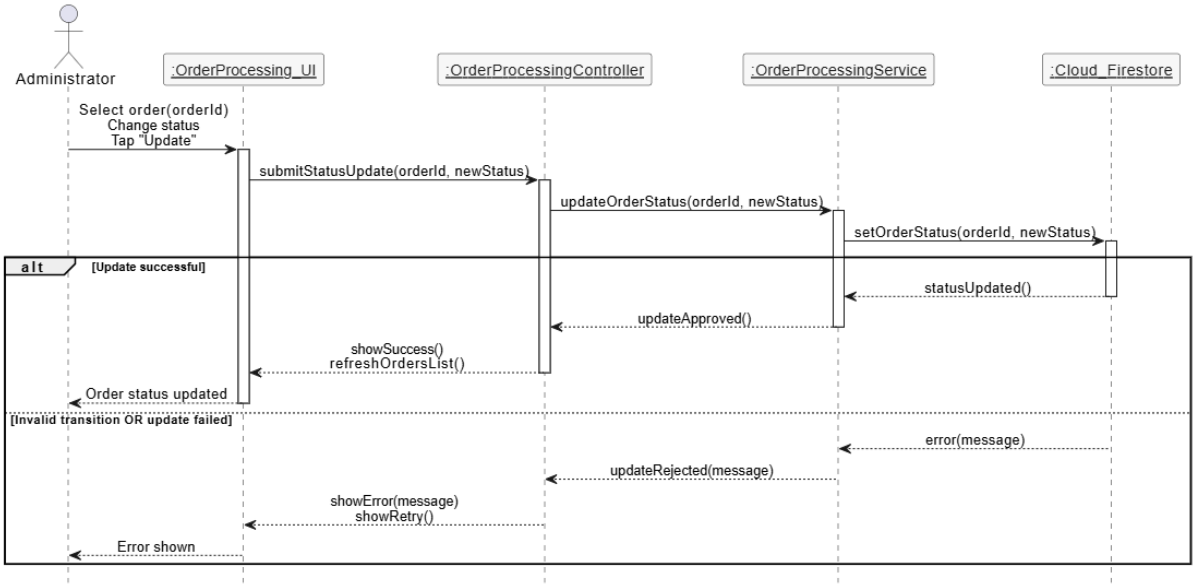

Figure 3.13: SD-11 Process Orders

3.6 CLASS DIAGRAM

Figure 3.14 presents the class diagram of the AR-enabled e-commerce system, showing the main classes, their attributes/operations, and relationships. UserAccount is generalized into Customer and Administrator. A Customer owns a Cart (containing CartItem entries) and places Orders (containing OrderItem entries) delivered to an Address. Each Order may have an associated Payment, whose behavior is controlled by PaymentMethod and PaymentStatus enumerations, while OrderStatus defines the order lifecycle. Products are organized by Category, enriched with ProductImage and optional ARModelAsset for AR preview, and tracked through InventoryStock for availability.

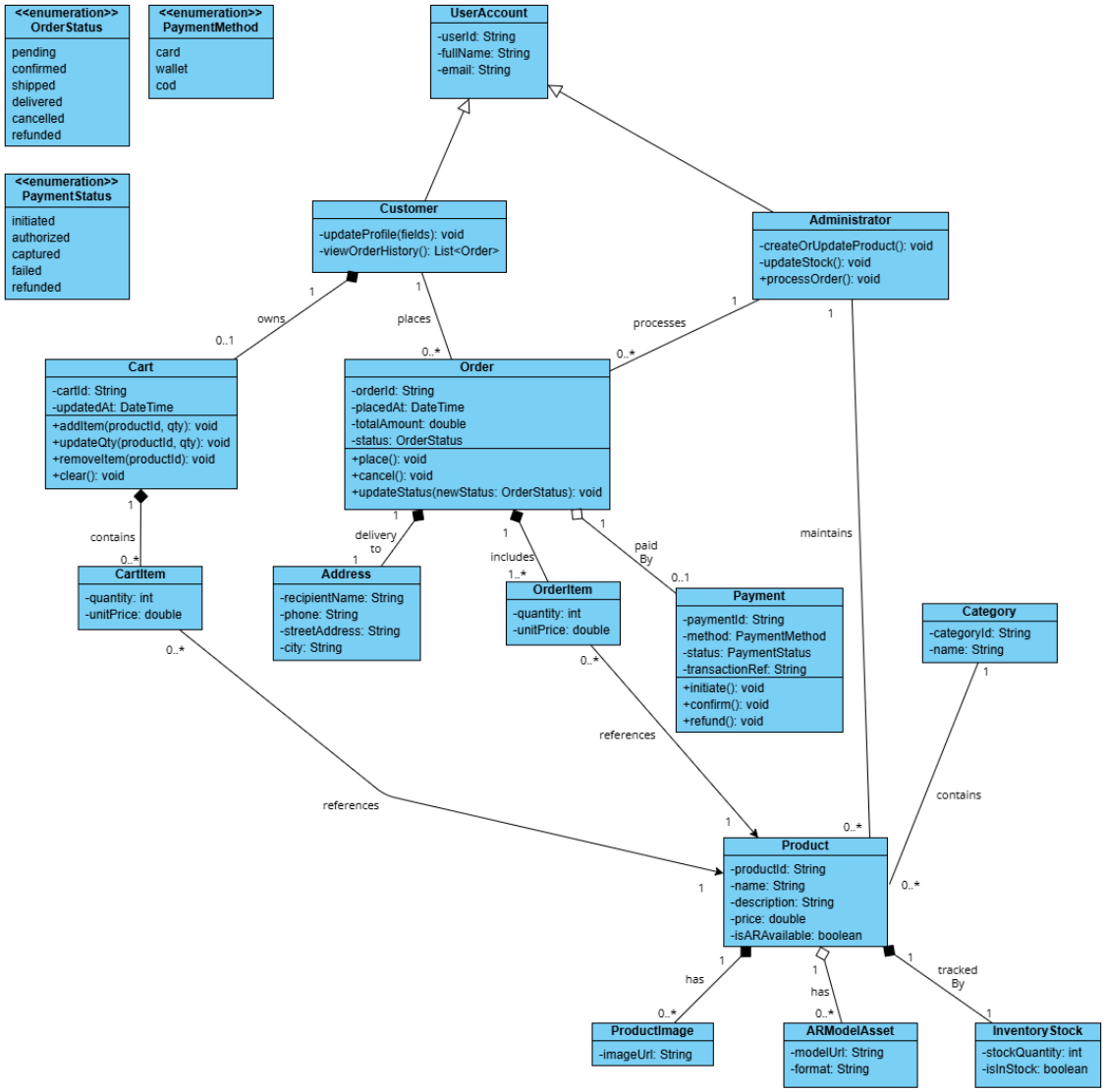

Figure 3.14: Class Diagram

CHAPTER # 4

SOFTWARE TEST DOCUMENTATION (STD)

4.1 INTRODUCTION

This chapter defines the testing strategy for the E-Commerce Platform with Augmented Reality, ensuring the application is reliable, correct, and usable before final deployment. Testing focuses on verifying the end-to-end shopping flow (authentication to catalog to product details to cart to checkout to order history), validating the AR preview behavior on supported devices, and confirming that data integrity and access control are preserved across Firebase services.

4.1.1 System Overview

The proposed system is a Flutter-based mobile application with two roles: Customer and Administrator. Customers can register, browse and search products, view product details, use AR preview where available, manage the cart, place orders, and view order history. Administrators maintain the product catalog, manage product media (images and 3D models), update stock, and process orders by updating their status.

The system uses Firebase Authentication for user accounts, Cloud Firestore for application data, and Firebase Storage for product images and 3D assets. AR preview is supported through ARCore/ARKit via a Flutter AR library, while payments target an online gateway with a fallback to COD/simulated payment if required.

4.1.2 Test Approach

Testing will be carried out using both automated and manual methods. Automated tests will cover core logic (validation, cart totals, stock updates, order status changes), Firebase integration (auth and database operations), and key user flows such as signup, browsing, checkout, and order history. Manual testing will focus on AR preview behavior on real AR-supported devices, along with general usability and error handling. Basic non-functional checks will also be performed for performance (screen loading/response) and security (role-based access and user data isolation).

4.2 TEST PLAN

4.2.1 Features to be Tested

The following features of the AR-enabled e-commerce system will be tested:

- Customer Registration and Login.
- Browse Product Catalog (including Search and Filter).
- View Product Details (including AR availability).
- AR Product Preview (device supported + model available cases).
- Shopping Cart Management (add/update/remove items, totals).
- Checkout and Place Order (validation + order creation).
- View Order History and Order Status.
- Admin Product Management (add/update/delete products).
- Admin Media Management (upload images and 3D models).
- Admin Inventory/Stock Updates.

- Admin Order Processing (view orders and update status).

4.2.2 Features not to be Tested

The following items are outside the scope of testing for this phase:

- AR preview on devices that do not support ARCore/ARKit.
- Highly realistic rendering checks (perfect lighting/shadows). Testing will focus on correct placement, scaling behavior, and stability.
- Third-party payment gateway certification/testing (PCI-level validation). If payment integration is incomplete, checkout will be validated using COD/simulated payment.

4.2.3 Testing Tools and Environment

Testing Tools:

- Flutter test for unit and widget tests.
- Flutter integration test for end-to-end flows on emulator/device.
- Firebase Emulator Suite or a dedicated Firebase test project for safe database/auth testing.
- Android Studio tools for device logs and debugging during AR testing.

Testing Environment:

- **Hardware:** Android/IOS smartphone (AR-supported for AR tests). Laptop/Desktop for development and admin testing.
- **OS:** Windows 10/11 (development), Android / IOS (runtime testing).
- **Framework/Platform:** Flutter (Dart), Firebase (Auth, Firestore, Storage).
- **Network:** Wi-Fi / 4G for catalog and media loading tests.

4.3 TEST CASES

4.3.1 TC-1: Registration Validation (Valid/Duplicate/Invalid)

Table 4.1 presents the test case for customer registration. It verifies that the system creates an account when valid details are provided and rejects invalid inputs such as missing fields, incorrect email format, weak password, or duplicate email. It also checks that clear feedback is shown to the user.

Test Case: TC-1	**Registration Validation (Valid/Duplicate/Invalid)**
Purpose	Verify that account creation accepts valid data and rejects invalid or duplicate credentials.
Inputs	Valid: name, email (new), password (meet rules). Invalid: missing fields/weak password/invalid email. Duplicate: already-registered email.
Pass/Fail Criteria	Pass: account created for valid data. Meaningful error shown for invalid/duplicate data.

	Fail: invalid data accepted, or valid data rejected without clear feedback.
Expected Outputs	Success: customer account created and profile initialized. Failure: error message displayed and no account created.
Test Procedure	1. Open Sign-Up screen. 2. Submit valid details and verify account creation. 3. Submit invalid details and verify validation errors. 4. Submit duplicate email and verify duplicate handling.

Table 4.1: TC-01 Registration Validation (Valid/Duplicate/Invalid)

4.3.2 TC-2: Catalog Load + Search/Filter Accuracy

Table 4.2 presents the test case for catalog loading and search/filter behavior. It validates that the product list is fetched and displayed correctly and that search keywords and category filters return relevant results. It also covers (no results) and query failure cases with proper UI response.

Test Case: TC-2	**Catalog Load + Search/Filter Accuracy**
Purpose	Verify catalog loading and that search/filter return correct product results.
Inputs	Search keyword; category filter; empty keyword; combination of keyword + category.
Pass/Fail Criteria	Pass: correct products displayed for each query/filter. (no results) state appears when applicable. Fail: incorrect/irrelevant results or app fails to update list.
Expected Outputs	Success: Catalog loads and displays products that match the applied search/filter. (No results found) appear if the query returns to the empty results. Failure: An error message is shown and a retry option is available. No incorrect products are displayed.
Test Procedure	1. Open Catalog screen. 2. Verify initial product list loads. 3. Apply keyword search and verify results. 4. Apply category filter and verify results. 5. Apply both and verify results/empty state.

Table 4.2: TC-02 Catalog Load + Search/Filter Accuracy

4.3.3 TC-3: Product Detail Integrity (Price/Media/Stock/AR Flag)

Table 4.3 presents the test case for product detail correctness. It ensures that product information such as name, description, price, images, stock availability, and AR indicator matches the stored record. It also verifies that the AR option is only shown/enabled when the product has a valid 3D model reference.

Test Case: TC-3	**Product Detail Integrity (Price/Media/Stock/AR Flag)**
Purpose	Verify that product details show correct data including stock status and AR availability indicator.
Inputs	Select product (in-stock with images). Select product (out-of-stock). Select product (AR model available). Select product (no AR model).
Pass/Fail Criteria	Pass: details match stored data. Stock shown correctly. AR option enabled only when model exists (and device supports AR). Fail: mismatch in price/stock/media, or AR shown incorrectly.
Expected Outputs	Success: Product details show correct name, price, description, images, stock state, and AR availability indicator (AR enabled only when model exists). Failure: A clear error message is shown (failed to load details), and the user can retry or return to catalog.
Test Procedure	1. Open a product from catalog. 2. Verify price/description/images. 3. Verify stock indicator for in-stock/out-of-stock products. 4, Verify AR indicator for products with/without model reference.

Table 4.3: TC-03 Product Detail Integrity (Price/Media/Stock/AR Flag)

4.3.4 TC-4: AR Preview Launch (Permission/Device/Model Cases)

Table 4.4 presents the test case for AR preview. It confirms that AR launches on supported devices when permission is granted and the 3D model is available. It also verifies safe handling of failure scenarios such as permission denial, unsupported devices, or model loading errors without crashing the app.

Test Case: TC-4	**AR Preview Launch (Permission/Device/Model Cases)**
Purpose	Verify AR preview behavior for supported devices and proper handling of failure conditions.
Inputs	Product with valid 3D model. Camera permission (granted/denied). AR-supported vs non-supported device.
Pass/Fail Criteria	Pass: AR launches with model placement on supported device with permission. Clear error shown for denied permission/unsupported device/model load failure. Fail: crash, blank view, or misleading AR availability.
Expected Outputs	Success: AR view opens, camera feed starts, and the product model loads and can be placed in the environment.

	Failure: A clear message is shown for permission denied/unsupported device/model load failure and the user returns safely to product details.
Test Procedure	1. Select an AR-enabled product. 2. Tap (View in AR). 3. Test permission granted path and verify model placement. 4. Deny permission and verify message + safe return. 5. Repeat on unsupported device and verify AR blocked with reason.

Table 4.4: TC-04 AR Preview Launch (Permission/Device/Model Cases)

4.3.5 TC-5: Cart Operations (Add/Update/Remove + Totals + Stock Limit)

Table 4.5 presents the test case for shopping cart operations. It verifies adding items, updating quantities, removing items, and recalculating totals correctly. It also checks that stock limits are enforced and the user is notified when requested quantities exceed available stock.

Test Case: TC-5	**Cart Operations**
Purpose	Verify cart behavior, correct totals, and enforcement of stockQuantity limits.
Inputs	Add product to cart. Update quantity (within stock). Update quantity (exceeds stock). Remove item.
Pass/Fail Criteria	Pass: cart updates persist. Totals correct. Quantity cannot exceed stock. Fail: wrong totals, quantity exceeds stock, or cart state not saved.
Expected Outputs	Success: Cart item is added/updated/removed correctly and totals update accurately. Quantity is limited by available stock. Failure: If stock is exceeded or operation fails, the cart is not updated incorrectly, and an error/stock warning is shown.
Test Procedure	1. Add an in-stock product to cart. 2. Verify cart items and totals. 3. Increase quantity within stock and verify totals update. 4. Try quantity > stock and verify rejection message. 5. Remove item and verify cart updates.

Table 4.5: TC-05 Cart Operations

4.3.6 TC-6: Checkout Order Creation + Stock Recheck + Inventory Decrement

Table 4.6 presents the test case for checkout and order placement. It validates delivery input fields, order creation, cart clearing, and correct inventory updates during order confirmation. It also checks that the system blocks checkout when stock becomes insufficient and provides an appropriate error message.

Test Case: TC-6	**Checkout Order Creation + Stock Recheck + Inventory Decrement**
Purpose	Verify checkout validation, order creation, cart clearing, and atomic stock decrement behavior.
Inputs	Delivery details (valid/invalid). Payment option (gateway or COD/simulated). Cart with in-stock items. Cart with insufficient stock scenario.
Pass/Fail Criteria	Pass: valid checkout creates order, reduces stock, clears cart. Insufficient stock blocks order. Fail: order created without stock update, partial updates, or invalid delivery accepted.
Expected Outputs	Success: order ID shown. Order stored. Inventory updated. Cart cleared. Failure: validation/stock error shown. No order confirmed.
Test Procedure	1. Add items to cart. 2. Proceed to checkout and submit valid delivery details. 3. Confirm order and verify order record + stock decrement + cart cleared. 4. Repeat with invalid fields and verify validation errors. 5. Repeat when stock is insufficient and verify order is blocked.

Table 4.6: TC-06 Checkout Order Creation

4.3.7 TC-7: Order History Access Control + Listing

Table 4.7 presents the test case for order history and access control. It verifies that each customer can view only their own orders and sees correct order status values. It also checks that unauthorized access to another user's orders is not allowed.

Test Case: TC-7	**Order History Access Control + Listing**
Purpose	Verify that customers can view only their own orders and correct statuses.
Inputs	Logged-in customer. Orders belonging to same customer. Orders belonging to a different customer (negative test).
Pass/Fail Criteria	Pass: only own orders displayed. No access to other users' orders. Fail: other users' orders visible or query returns unauthorized data.
Expected Outputs	Success: Order history displays only the logged-in customer's orders with correct current status.

	Failure: Unauthorized orders are not displayed. Access is denied or data is hidden if an invalid request is made.
Test Procedure	1. Login as Customer A and Open Order History. 2. Verify only Customer A orders appear. 3. Attempt to access Customer B order (if testable) and verify access denied/hidden.

Table 4.7: TC-07 Order History Access Control + Listing

4.3.8 TC-8: Profile Update Validation + Persistence

Table 4.8 presents the test case for profile updates. It verifies that editable profile fields are validated and saved properly and remain consistent after reopening the screen. It also ensures that invalid values are rejected with clear error feedback.

Test Case: TC-8	**Profile Update Validation + Persistence**
Purpose	Verify profile editing validation and that updates persist correctly.
Inputs	Update name/phone/address fields (valid). Invalid inputs (empty required fields, invalid phone).
Pass/Fail Criteria	Pass: valid updates saved and shown on reload. Invalid updates rejected with message. Fail: invalid data saved or valid data did not persist.
Expected Outputs	Success: Profile updates are saved and shown correctly after refresh/reopen. Failure: Invalid inputs are rejected with a message and previous valid data remains unchanged.
Test Procedure	1. Open Profile screen. 2. Update valid fields and save. 3. Reopen Profile and confirm persistence. 4. Enter invalid values and confirm validation blocks save.

Table 4.8: TC-08 Profile Update Validation + Persistence

4.3.9 TC-9: Admin Product CRUD + Field Validation

Table 4.9 presents the test case for admin product management. It verifies product creation, updating, and deletion along with required field validation. It also checks that changes are reflected correctly in the customer catalog view.

Test Case: TC-9	**Admin Product CRUD + Field Validation**
Purpose	Verify admin can add/update/delete products and that required fields are enforced.
Inputs	Add product (valid fields). Add product (missing required fields). Update product (price/description/category). Delete product.
Pass/Fail Criteria	Pass: CRUD works for admin. Invalid product data rejected.

	Fail: invalid data accepted or CRUD fails without feedback.
Expected Outputs	Success: Admin can add/update/delete products and changes reflect in customer catalog. Failure: Invalid product data is rejected with an error and no incorrect product record is saved.
Test Procedure	1. Login as admin and open Product Management. 2. Create products with valid fields and verify them appear in catalog. 3. Try to create/update missing fields and verify validation errors. 4. Update existing products and verify changes. 5) Delete a product and verify removal.

Table 4.9: TC-09 Admin Product CRUD + Field Validation

4.3.10 TC-10: Admin Media Upload + Product Linking

Table 4.10 presents the test case for admin media management. It verifies uploading product images and 3D model assets and correctly linking them to a product record. It also ensures that images appear in product details and that AR becomes available only after a valid model is associated.

Test Case: TC-10	**Admin Media Upload + Product Linking**
Purpose	Verify that admin uploads media and correctly links it to a product for display and AR availability.
Inputs	Upload image (valid/invalid format). Upload 3D model (valid/invalid). Select target product.
Pass/Fail Criteria	Pass: media uploads succeed. Product updates with media references. Customer sees images. AR becomes available when model exists. Fail: broken links, media not visible, or AR enabled without valid model reference.
Expected Outputs	Success: Images and 3D model upload successfully and are linked to the product. Images appear in product details and AR becomes available after model linking. Failure: Unsupported/failed uploads are rejected with an error; broken links are not saved.
Test Procedure	1. Login as admin and select a product. 2. Upload product images and verify images appear on customer product detail screen. 3. Upload/associate 3D model and verify AR indicator becomes available. 4. Attempt invalid upload and verify rejection message.

Table 4.10: TC-10 Admin Media Upload + Product Linking

4.3.11 TC-11: Admin Stock Update Rules (Non-negative + Reflected)

Table 4.11 presents the test case for inventory/stock management. It verifies that stock updates are accepted only for valid non-negative values and that setting stock to zero reflects an out-of-stock state. It also checks that customer cart, and checkout flows respect updated stock levels.

Test Case: TC-11	**Admin Stock Update Rules**
Purpose	Verify stock updates are validated and reflected in customer views/cart constraints.
Inputs	Set stockQuantity to valid integer. Set stockQuantity to negative/non-numeric. Set stock to 0 (out-of-stock).
Pass/Fail Criteria	Pass: stock updates saved. Negative values blocked. Out-of-stock reflected correctly. Fail: negative stock saved or customer can exceed stock in cart.
Expected Outputs	Success: Stock updates save correctly. Stock = 0 shows out-of-stock and cart/checkout respects the updated stock. Failure: Negative/invalid stock values are rejected. Stock state does not become inconsistent.
Test Procedure	1. Login as admin and update product stock to a valid value. 2. Verify customers sees correct stock status. 3. Set stock to 0 and verify out-of-stock behavior. 4. Try negative stock and verify update is rejected.

Table 4.11: TC-11 Admin Stock Update Rules

4.3.12 TC-12: Admin Order Status Update + Customer Visibility

Table 4.12 presents the test case for admin order processing. It verifies that the administrator can update order status across the defined lifecycle and that the latest status persists. It also confirms that customers can see updated statuses in their order history.

Test Case: TC-12	**Admin Order Status Update + Customer Visibility**
Purpose	Verify admin can update order status and customer sees the updated status in order history.
Inputs	Select existing order. Change status (pending to confirmed to shipped to delivered). Optional cancellation/refund states if supported in the build.
Pass/Fail Criteria	Pass: status update persists. Customer sees updated status. Fail: status does not persist, inconsistent states, or customer view not updated.

Expected Outputs	Success: Admin status update persists and the customer sees the updated status in order history. Failure: Invalid/failed status updates do not apply and an error is shown. Customer view remains consistent.
Test Procedure	1. Place an order as customer. 2. Login as admin and open Order Management. 3. Update status step-by-step and verify save confirmation. 4. Login as customer and verify the new status appears in Order History.

Table 4.12: TC-12 Admin Order Status Update

REFERENCES

Ref No.	Source	Link
1	IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications	http://www.math.uaa.alaska.edu/~afkjm/cs401/IEEE830.pdf
2	R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed., McGraw-Hill	https://mlsu.ac.in/econtents/16_EBOOK-7th_ed_software_engineering_a_practitioners_approach_by_roger_s._pressman_.pdf
3	BigCommerce, "Augmented Reality in Ecommerce: How Does It Work?"	https://www.bigcommerce.com/articles/ecommerce/ecommerce-augmented-reality/
4	IKEA, "IKEA Place app launched to help people virtually place furniture at home," Newsroom	https://www.ikea.com/global/en/newsroom/innovation/ikea-launches-ikea-place-a-new-app-that-allows-people-to-virtually-place-furniture-in-their-home-170912/
5	Amazon, "View in your room" / AR View feature in Amazon Shopping App	https://www.amazon.com/
6	Houzz, "Houzz App Upgrades 3D Augmented Reality Tool with ARKit to Help You Design and Shop," Press Release	https://www.houzz.com/press/398/Houzz-App-Upgrades-3D-Augmented-Reality-Tool-with-ARKit-to-Help-You-Design-and-Shop
7	E-Commerce with Augmented Reality (AR)	https://www.freepik.com/free-vector/e-commerce-augmented-reality-infographic-flat_elements